



منصة صرح — توثيق المشروع

السجل العقاري الليبي + إصدار الهوية الرقمية

الرؤية الليبية للاتصالات والتقنية — LVCT · 2026

الفصل الأول — المقدمة

1.1 خلفية المشروع

تُعدّ ملكية العقارات في ليبيا من أكثر الملفات حساسية، إذ يعتمد التوثيق العقاري تاريخياً على **الصكوك الورقية** و«الكروكي» المرسوم يدوياً، ما أدّى إلى مشكلات متراكمة: ضياع المستندات، صعوبة التحقق من صحتها، **ازدواجية التسجيل**، وتداخل حدود الأراضي، وبيع الأرض الواحدة لأكثر من شخص. كما لا توجد منظومة هوية رقمية وطنية موحّدة يمكن الاستناد إليها لربط المالك بعقاره ربطاً موثقاً.

منصّة **صرح** مشروع من الرؤية الليبية للاتصالات والتقنية (LVCT) يهدف إلى رقمنة السجل العقاري وإصدار هوية رقمية للمواطن، مع ضمانات تقنية قوية ضد التزوير والاستنساخ، وبناء أرشيف مكاني (جغرافي) دقيق لحدود الأراضي.

1.2 تعريف المشكلة (Problem Statement)

1. غياب سجل عقاري رقمي موثوق: الاعتماد على الورق يجعل التحقق بطيئاً وعرضة للتلاعب.
2. ازدواجية الملكية وتداخل الحدود: لا توجد آلية آلية تمنع تسجيل أرض في موقع أرض مسجّلة سابقاً، ما يفتح الباب لبيع الأرض لأكثر من مشتري.
3. ضعف التحقق العام: لا يستطيع طرف ثالث (مصرف، محكمة، مشتري) التأكد بسهولة من صحة صكّ.
4. انعدام الهوية الرقمية: لا توجد بطاقة هوية رقمية مقاومة للاستنساخ تربط المواطن بسجلّه.
5. غياب أثر تدقيقي: لا سجلّ غير قابل للتعديل يوثّق من فعل ماذا ومتى.

1.3 أهداف المشروع (Objectives)

- بناء سجل عقاري رقمي يدير دورة حياة العقار من التقديم حتى إصدار الصكّ.
- اعتماد المساحة المحسوبة من المضلّع الجغرافي بدل الطول×العرض، والتحقق منها على الخادم.
- منع/كشف تداخل الأراضي وازدواجية الإحداثيات لحماية الملكية.
- إصدار هوية رقمية على شريحة NFC مقاومة للاستنساخ (NTAG 424 DNA).
- إصدار صكوك PDF موقّعة رقمياً (PAdES) مع رمز QR للتحقق العام.

- توفير بوابة تحقق عامة للتأكد من صحة الصكوك دون كشف بيانات شخصية.
- ضمان سجل تدقيق غير قابل للتعديل لكل عمليات الكتابة.
- بناء كل الواجهات بالعربية أولاً (RTL).

1.4 نطاق المشروع (Scope)

داخل النطاق

- إدارة المواطنين والهوية الرقمية وبطاقات NFC.
- تقديم العقارات ورسم الحدود، ومراجعتها واعتمادها، وإصدار الصكوك.
- الخرائط المساحية (للموظف والعام) وكشف التداخل والنزاعات.
- التحقق العام من الصكوك.
- لوحة إدارة وتقارير، وسجل تدقيق.

خارج النطاق (حالياً)

- التكامل مع منظومة هوية وطنية رسمية (مُهيئاً مستقبلاً عبر حقل legacy_national_no).
- شبكة بلوكتشين إنتاجية حقيقية (مستخدَم حالياً عبر طبقة بديلة Stub قابلة للاستبدال).
- المدفوعات الإلكترونية والرسوم.

1.5 أصحاب المصلحة والمستخدمون (Stakeholders & Users)

الدور	الوصف
المواطن	يقدم طلبات تسجيل العقار، يرسم الحدود، يتابع حالته، يملك هوية رقمية.
موظف السجل (registry_officer)	يراجع الطلبات في منطقته، يدقق المستندات والحدود.
المراجع (reviewer)	مراجعة وتدقيق إضافي ضمن المنطقة.
مدير الإدارة (department_manager)	الاعتماد النهائي وإصدار الصكّ/الرخصة.
مُصدّر الهوية (id_issuer)	يصدر بطاقات الهوية الرقمية على شرائح NFC.
المدير العام (super_admin)	صلاحيات كاملة على النظام.
المدقق (auditor)	اطّلاع على سجل التدقيق والتقارير.
العموم (Public)	التحقق من صحة الصكوك عبر بوابة عامة دون تسجيل دخول.

1.6 مخرجات المشروع (Deliverables)

- خدمة خلفية (API) بـ ASP.NET Core 8.
- تطبيق ويب موحد بـ Angular 21 لكل الأدوار.
- تطبيق محمول بـ Flutter للمواطن.
- قاعدة بيانات SQL Server مع طبقة جغرافية وسجل تدقيق.
- توثيق المشروع (هذا المجلّد).

1.7 منهجية العمل (Methodology)

اتُبع أسلوب تطوير تدريجي مرحلي (Incremental/Agile) عبر مراحل مرقّمة (Phase PROMPTS.md → Phase 12) موثّقة في PROMPTS.md ، تبدأ من تجهيز البنية والمخطط، مروراً بالمصادقة والوحدات، وصولاً إلى التحقق العام والتدعيم الأمني. كل ميزة تُطوّر على فرع مستقل وتُدمج عبر طلب دمج (Pull Request) بعد المراجعة (انظر سجل المستودع).

الفصل الثاني — تحليل المتطلبات

2.1 الأدوار والصلاحيات (Roles)

الصلاحيات لا تُفحص بنصوص حزمة في الكود، بل عبر خريطة صلاحيات JSON مخزنة على `officers.permissions`، وتُفرض في الواجهة عبر دالة موحدة لتحديد ما يصل إليه كل دور.

الدور	أبرز الصلاحيات
citizen	تقديم عقار، متابعة طلباته، عرض هويته الرقمية.
registry_officer	مراجعة طلبات منطقتة، تدقيق المستندات والحدود، تعديل الحدود.
reviewer	مراجعة/تدقيق ضمن المنطقة.
department_manager	الاعتماد النهائي وإصدار الصك/سكّ الرخصة.
id_issuer	إصدار بطاقات الهوية الرقمية (NFC).
super_admin	كل الصلاحيات + إدارة المستخدمين.
auditor	الاطلاع على سجل التدقيق والتقارير.

2.2 المتطلبات الوظيفية (Functional Requirements)

وحدة المواطنين والهوية (Citizens & Digital ID)

- FR-1: تسجيل/إدارة بيانات المواطن مع الاحتفاظ بحقل `legacy_national_no` لإعادة الربط مستقبلاً.
- FR-2: إصدار هوية رقمية برقم بصيغة `LY-XX-YYYY-NNNNNN-D`.
- FR-3: إصدار بطاقة (NFC (NTAG 424 DNA مرتبطة بالهوية مع PIN وصلاحية.
- FR-4: تسجيل دخول المواطن برقم الهوية الرقمية + PIN، والموظف بالبريد + كلمة المرور.

وحدة العقارات (Properties)

- FR-5: تقديم عقار برسم مضلع الحدود على الخريطة (3 نقاط على الأقل).
- FR-6: اشتقاق المساحة من المضلع وإعادة حسابها على الخادم والتحقق ضمن $\pm 5\%$.

- FR-7: إلزامية المرفقات: صورة موقع + شهادة كروكي على الأقل، وإلا يُرفض الطلب.
- FR-8: منع تطابق الإحداثيات (مركز القطعة) لعقارين معتمدين (رفض صارم).
- FR-9: كشف تداخل الأراضي وتمييز حالته: «خلل في الملكية» عند التداخل مع عقار معتمد، و«تضارب في الموقع» عند تداخل عقارين غير معتمدين.
- FR-10: مراجعة الطلب (اعتماد/رفض/طلب توضيح) مع ملاحظة ورقم قرار.
- FR-11: تعديل/إعادة رسم حدود العقار من قبل الموظف.

الخرائط والتحقق (Map & Verify)

- FR-12: خريطة مساحية للموظف تعرض كل قطع منطقته مع تمييز الحالة (سليم/نزاع/قيد المراجعة/تضارب).
- FR-13: خريطة عامة تعرض القطع الصادرة صكوكها فقط، دون أي بيانات شخصية.
- FR-14: بوابة تحقق عامة عبر رمز QR تتحقق من توقيع الصكّ.

الإدارة والتدقيق (Admin & Audit)

- FR-15: لوحة إدارة وتقارير.
- FR-16: تسجيل كل عملية كتابة في سجل تدقيق غير قابل للتعديل.
- FR-17: إشعارات داخلية للمراجعين عند ورود طلب/تضارب.

2.3 المتطلبات غير الوظيفية (Non-Functional Requirements)

المعرّف	المتطلب
NFR-1 (لغة)	كل الواجهات بالعربية RTL؛ اللاتينية للرموز والمعرّفات التقنية فقط.
NFR-2 (أمان)	JWT (HS256) بلا بيانات شخصية، bcrypt لكلمات المرور، CORS مقيّد، تحديد معدّل على <code>/auth/*</code> و <code>/properties</code> .
NFR-3 (مقاومة الاستنساخ)	بطاقات DNA 424 NTAG مع رسالة SUN وعدّاد متدحرج يُتّحقق منه على الخادم.
NFR-4 (سلامة المستندات)	كل صكّ موقع PAdES ويحمل QR للتحقق.
NFR-5 (سلامة التدقيق)	سجل التدقيق إلحاقى فقط (لا UPDATE/DELETE).
NFR-6 (الدقة المكانية)	المساحة من المضلّع، والتحقق من التداخل والمركز عبر <code>geography</code> .
NFR-7 (الأداء)	ترقيم بالمؤشّر (cursor) لا الإزاحة على الجداول الكبيرة.
NFR-8 (قابلية النقل)	تهيئة قاعدة البيانات عبر ترحيلات EF تعمل على أي جهاز فيه .NET 8 SDK + SQL Server.
NFR-9 (إعادة الإصدار)	إمكانية الهجرة لهوية وطنية مستقبلية دون فقد بيانات.

2.4 حالات الاستخدام (Use Cases)

المعرّف	الفاعل	الحالة
UC-1	مواطن	تسجيل الدخول بالهوية + PIN
UC-2	مواطن	تقديم عقار جديد ورسم حدوده وإرفاق المستندات
UC-3	مواطن	متابعة حالة الطلب
UC-4	موظف سجل	فتح طابور المراجعة واختيار طلب
UC-5	موظف سجل	فحص الخريطة والتأكد من عدم وجود تداخل
UC-6	موظف سجل	اعتماد/رفض/طلب توضيح
UC-7	مدير الإدارة	الاعتماد النهائي وإصدار الصكّ
UC-8	مُصدّر هوية	إصدار بطاقة NFC للمواطن
UC-9	العموم	التحقق من صكّ عبر QR
UC-10	مدقق	الاطّلاع على سجل التدقيق

مثال مفصّل — UC-2 «تقديم عقار»

1. يسجّل المواطن الدخول ويفتح شاشة «عقار جديد».
2. يرسم مضلع الحدود على الخريطة (تظهر الأراضي المسجّلة كخلفية رمادية لتفادي الرسم فوقها).
3. يُدخل النوع والمنطقة والمساحة، ويرفق صورة الموقع والكروكي.
4. عند الإرسال: يتحقق الخادم من المساحة، ويرفض تطابق المركز، **ويفحص التداخل**؛ فإن وُجد يُنبّه المراجعون ويوسّم العقار بحالة التضارب المناسبة.
5. يُنشأ طلب تسجيل بحالة `pending` ويُشعر مراجعو المنطقة.

2.5 قصص المستخدمين (User Stories) — أمثلة

- «كمواطن، أريد رسم حدود أرضي على الخريطة بدقّة كي تُحسب مساحتها تلقائياً.»
- «كموظف سجل، أريد رؤية كل القطع المجاورة على الخريطة كي أكتشف أي تداخل قبل الاعتماد.»
- «كمشتري، أريد التحقق من صحة صكّ عبر مسح رمز QR دون الحاجة لحساب.»

2.6 القيود (Constraints)

مستمدة من وثيقة قيود البناء (CLAUDE.md)، وهي قيود غير قابلة للتفاوض:

1. العربية أولاً RTL لكل الواجهات.
2. الاحتفاظ بـ legacy_national_no لإعادة إصدار الهوية مستقبلاً.
3. لا عقارين معتمدين بنفس المركز؛ التداخل يثير تحذيراً للمراجع (لأن الصكوك الورقية القديمة قد تتعارض شرعياً) — وقد طُور إلى تمييز حالي «خلل في الملكية» و«تضارب في الموقع».
4. بطاقات DNA 424 NTAG فقط مع SUN وعدّاد متدحرج.
5. صكوك موقعة PAdES مع QR.
6. سجل التدقيق إلحاق فقط.
7. لا فحص أدوار بنصوص حرّة — عبر خريطة الصلاحيات.
8. المساحة من المضلع لا من الطول×العرض.
9. كل تقديم عقار يحمل دليلاً (صورة + كروكي).

الفصل الثالث — البنية والتصميم

3.1 المعمارية العامة (System Architecture)

يتبع النظام نمط «وحدات مترابطة في خدمة واحدة» (Modular Monolith) للخلفية، مع عملاء متعددين:



3.2 وحدات النظام (Modules)

المسؤولية	الوحدة
توقيع/تحقق (JWT (HS256), bcrypt، الصلاحيات.	Auth
بيانات المواطنين.	Citizens
إصدار الهوية الرقمية وتشفير بطاقات (SUN/AES- NTAG 424 DNA (CMAC).	DigitalIdCards + Nfc
تقديم/مراجعة العقارات، التحقق المكاني، كشف التداخل، تعديل الحدود.	Properties
تغذية الخرائط (الموظف/العامة) كـ GeoJSON.	Map
الحجوزات والنزاعات القانونية على العقارات.	Disputes
سير الاعتماد وإصدار/توقيع الصكوك (PAdES) وسكّ الرخصة.	Workflow
التحقق العام + بثّ سكّ PDF.	Verify
تكامل ACA-Py لإصدار الاعتمادات القابلة للتحقق.	Ssi
طبقة بديلة (Stub) لسكّ سجل الملكية + IPFS، قابلة للاستبدال.	Blockchain
اعتراض عمليات الكتابة وتسجيلها.	Audit
الإشعارات الداخلية.	Notifications

3.3 تصميم قاعدة البيانات (Database Design)

اصطلاحات: أسماء snake_case، مفاتيح أولية (UNIQUEIDENTIFIER DEFAULT NEWID()) (عدا audit_log فهو BIGINT IDENTITY للترتيب)، أعمدة الوقت DATETIMEOFFSET(3)، الحذف الناعم عبر is_active = 0، وال ENUM عبر CHECK (col IN (...)).

أبرز الكيانات والعلاقات

```

citizens 1—* properties (owner_citizen_id)
citizens 1—* digital_id_cards (citizen_id)
digital_id_cards 1—1 nfc_card_secrets
properties 1—* property_documents (صور + كروكي - دليل إلزامي)
properties 1—* registration_requests
properties 1—* property_disputes (حجز/نزاع نشط يمنع البيع/السك)
properties 1—* property_nfts (سجل الملكية المسكوك)
regions (الشعبيات) 1—* properties
officers *—1 offices / regions
audit_log (إلحاق فقط) —> جميع عمليات الكتابة *
```

حالات العقار (Property Status)

ثم `draft` → `pending` → `under_review` → (`approved` | `rejected` | `needs_clarification`)
`approved` → `minted` → `transferred` مع `frozen` للتجميد الإداري. بالإضافة إلى حالة تضارب
مشتقة (غير مخزنة) تُحسب جغرافياً: «خلل في الملكية» أو «تضارب في الموقع».

3.4 تدفقات العمل (Workflows)

مسار تسجيل عقار (Submit → Verify)

المواطن يرسم المزلّج ويرفق الدليل

|

▼

تحقق المساحة ($\pm 5\%$) + رفض تطابق المركز + كشف التداخل — `POST /properties`

|

▼

إشعار مرآجي المنطقة $\rightarrow$ `registration_requests = pending`

|

▼

الموظف يراجع (خريطة كل القطع + المستندات + تحذير التضارب)

|



|

▼

(NFT) سكّ الرخصة $\rightarrow$ QR + (PAdES) مدير الإدارة: اعتماد نهائي $\rightarrow$ توقيع الصكّ

|

▼

تحقق عام — `verify.sarh.ly/{deed_id}`

3.5 نموذج الأمان (Security Design)

الطبعة	الآلية
المصادقة	JWT موقع HS256؛ لا يحمل سوى <code>sub</code> , <code>role</code> , <code>citizen_id / officer_id</code> , <code>exp</code> — بلا بيانات شخصية.
كلمات المرور	bcrypt (BCrypt.Net-Next) بمعامل تكلفة.
الصلاحيات	خريطة JSON على <code>officers.permissions</code> + سمة <code>[OfficerOnly]</code> لكل دالة.
عزل البيانات	RLS على الجداول الحساسة، ونطاق المنطقة للموظفين.
بطاقات NFC	NTAG 424 DNA: رسالة SUN، فكّ PICC بـ AES-CBC، اشتقاق مفتاح جلسة، تحقق CMAC بزمان ثابت، عداد متدرج يمنع إعادة التشغيل.
الصكوك	توقيع PAdES (PKCS#7) + رمز QR للتحقق؛ أي عبث يُبطل التوقيع.
التدقيق	<code>audit_log</code> إلحاق عبر مُحفّزات <code>INSTEAD OF UPDATE/DELETE</code> .
الحدود	geography : رفض تطابق المركز (فهرس فريد مُرشّح)، وكشف التداخل بمساحة التقاطع.
الشبكة	CORS مقيد بالواجهات المعروفة، وتحديد معدّل على المسارات الحساسة.

3.6 تصميم الواجهات (UI/UX Design)

- تطبيق Angular موحد يضم كل الأدوار خلف توجيه قائم على الصلاحيات.
- رموز تصميم (Design Tokens) للهوية البصرية بدل مكتبة جاهزة، مع دعم RTL كامل.
- ألوان الهوية: الأساسي #0F172A، الذهبي #F97316، الأحمر #DC2626، الأخضر #0891B2.
- مكونات مستقلة (standalone) مع OnPush و Signals للحالة التفاعلية.
- ألوان الخريطة المساحية المشتقة: سليم ● نزع ● قيد المراجعة ● ملكية عامة، مع حدّ أحمر متقطع لحالات التضارب.

3.7 الطبقة الجغرافية (Geospatial Design)

- النوع (SRID 4326) geography لتخزين المضلّعات.
- المساحة عبر STArea() على الخادم (لا الطول×العرض)، وتقدير على العميل بصيغة الفائص الكروي.

- **المركز** عبر `EnvelopeCenter()` ومقارنته بـ `STEquals` لمنع التكرار.
- **التداخل** عبر `STIntersects` + قياس **مساحة التقاطع** `STIntersection().STArea()` ،
باعتبار العتبة $< 1\text{م}^2$ لتجاهل الأراضي المتجاورة التي تتشارك حدّاً فقط (مساحة تقاطعها = صفر).

منصة صرح (Sarh) — تقرير مرحلتي التنفيذ والاختبار

المشروع: منصة صرح — السجل العقاري الليبي + إصدار الهوية الرقمية
الجهة المالكة: الرؤية الليبية للاتصالات والتقنية (LVCT)
طبيعة النظام: نظام ويب + خدمة خلفية + تطبيق محمول، يعمل بالعربية أولاً (RTL).

4. مرحلة التنفيذ (Implementation Phase)

1.4 بيئة العمل وأدوات التطوير (Development Environment & Tools)

في هذا القسم تُسرد الخيارات التقنية التي بُني عليها النظام فعلياً، مع تبرير كل اختيار.

أ) لغات البرمجة (Programming Languages)

اللغة	الإصدار	موضع الاستخدام	سبب الاختيار
#C	12 (على NET 8.)	الخدمة الخلفية (Backend API)	لغة مُدارة قوية الأنواع (strongly-typed)، أداء عالٍ، ودعم أصيل لـ SQL Server والتشفير (System.Security.Cryptography) المطلوب لبطاقات NFC وتوقيع الصكوك.
TypeScript	5.4.5	تطبيق الويب (Angular)	إضافة الأنواع الثابتة إلى JavaScript تقلّل الأخطاء في واجهة كبيرة متعددة الأدوار، وتشارك نماذج البيانات مع الخلفية عبر حزمة shared-types.
Dart	3.4	تطبيق الهاتف (Flutter)	لغة Flutter الرسمية؛ ترجمة أصلية (AOT) لأداء قريب من الأصلي على أندرويد/iOS.
T-SQL	SQL Server 2019/2022	قاعدة البيانات (مخططات، إجراءات، مُحفّزات)	تنفيذ منطق حسّاس مكانياً (geography) ومُحفّزات سجل التدقيق غير القابل للتعديل مباشرة داخل المحرك.
SCSS / HTML5	—	تنسيق واجهات الويب	رموز تصميم (design tokens) للهوية البصرية بدل مكتبة جاهزة، مع دعم RTL كامل.

ب) أطر العمل والمكتبات (Frameworks & Libraries)

الخلفية (Backend):

- **ASP.NET Core 8** — إطار خدمة الويب و REST API. يوفّر التوجيه، حقن الاعتماديات (DI)، و«الوسطاء» (middleware) للمصادقة وتحديد المعدّل (rate limiting).
- **Entity Framework Core 8 + Microsoft.Data.SqlClient** — الوصول إلى البيانات. نستخدم EF للاستعلامات العادية، وننزل إلى SqlCommand المباشر للعمليات المكانية (geography) التي لا يجزّدها EF.
- **BCrypt.Net-Next** — تجزئة كلمات المرور (مقاومة لهجمات القوة الغاشمة عبر معامل التكلفة).
- **System.IdentityModel.Tokens.Jwt** — توقيع/تحقق رموز JWT بخوارزمية HS256.

الويب (Web):

- **Angular 21** — تطبيق صفحة واحدة (SPA) موحّد يضم جميع الأدوار (مواطن، موظف سجل، مُصدّر هوية، مدير، مدقق، تحقق عام) خلف توجيه قائم على الصلاحيات.

- **Angular Signals** — إدارة الحالة التفاعلية الحديثة (`signal` , `computed`) بدل RxJS الثقيل في معظم المكوّنات.
- **Leaflet** — عرض الخرائط ورسم مضلّعات حدود الأراضي (بديل خفيف ومفتوح المصدر).

الهاتف (Mobile):

- **Flutter 3.22+** — إطار واجهة المستخدم متعدّد المنصّات.
- **Riverpod 2.x** — إدارة الحالة (نمط شبيه بـ MVVM).
- **go_router** — التوجيه المعتمد على المسارات.
- **flutter_nfc_kit** — قراءة بطاقات DNA 424 NTAG.
- **mapbox_maps_flutter** — الخرائط ورسم الحدود على الهاتف.

الهوية ذاتية السيادة (SSI):

- **Hyperledger Aries Cloud Agent Python (ACA-Py) v0.12** — إصدار «الاعتمادات القابلة للتحقق» (Verifiable Credentials) بطريقة `did:sov`.

ج) قاعدة البيانات (Database) وإعدادها

- **المحرّك:** SQL Server 2019/2022 محلياً.
- **النوع المكاني** (SRID 4326) `geography` لتخزين مضلّعات الأراضي وحساب المساحات والتداخل (`STArea` , `STIntersects` , `STIntersection` , `EnvelopeCenter`).
- **فهرس النص الكامل (Full-Text, عربي)** للبحث في أسماء المواطنين والعقارات.
- **سجل تدقيق (audit_log) إلحاقى فقط** عبر مُحفّزات `INSTEAD OF UPDATE/DELETE` تمنع أي تعديل أو حذف.
- **التهيئة الأساسية = ترحيلات (EF Core (EF Migrations):** كل ملف T-SQL مرّقم (`infra/mssql/migrations/000...044.sql`) مضمّن داخل التجميع كـ `EmbeddedResource` ، ويطبّقه `EfDatabaseBootstrapper` تلقائياً عند الإقلاع، ويسجّله في `__EFMigrationsHistory` . ميزة هذا الأسلوب أنه يعمل على أي جهاز فيه .NET 8 SDK. وخادم SQL Server، دون الحاجة إلى `sqlcmd` أو تشغيل ملفات يدوياً.

التشغيل: `pnpm db:migrate:ef` (تهيئة ثم خروج) أو `pnpm db:reset:ef` (إعادة بناء كاملة).

د) بيئات التطوير المتكاملة (IDEs)

الأداة	الاستخدام
Visual Studio 2022 / VS Code	تطوير الخلفية .NET / C#.
VS Code	تطوير الويب (Angular/TypeScript) وملفات SCSS وSQL.
Android Studio	تطوير ومحاكاة تطبيق Flutter.
SQL Server Management Studio (SSMS)	فحص قاعدة البيانات والاستعلامات المكانية.

هـ) أدوات إدارة النسخ (Version Control)

- **Git** نظام التحكم بالإصدارات، و**GitHub** للاستضافة والمراجعة.
- **سير العمل بالفروع (Branching):** فرع رئيسي `main` مستقر، وفروع ميزات وصفية مثل `feature/idissuer-nav-and-card-list-actions`.
- **طلبات الدمج (Pull Requests):** كل ميزة تُدمج عبر PR تتم مراجعته (انظر سجل المشروع: PR #12-#16).
- **خطافات (Git Hooks):** `core.hooksPath = .githubhooks` تُفعّل تلقائياً عبر سكربت `prepare` للتحقق قبل الالتزام.

2.4 البنية البرمجية وهندسة النظام (System Architecture & Coding Structure)

أ) هيكل المجلدات (Folder Structure)

sarh/

└─ apps/

```
| └─ api-dotnet/ # ASP.NET Core 8 الخدمة الخلفية
|   | └─ Program.cs # نقطة الإقلاع + تسجيل الخدمات والوسطاء
|   | └─ Auth/ # JWT + bcrypt + مصادقة الصلاحيات
|   | └─ Citizens/ # وحدة المواطنين
|   | └─ Properties/ # (الخدمة + DTOs + GeoJSON) وحدة العقارات
|   |   | └─ PropertiesService.cs
|   |   | └─ PropertyDtos.cs
|   |   | └─ GeoJsonPolygon.cs
|   | └─ DigitalIdCards/ # إصدار بطاقات الهوية الرقمية
|   | └─ Nfc/ # NTAG 424 DNA (SUN + AES-CMAC) تشفير
|   | └─ Map/ # تغذية الخريطة (مضلعات للموظف/العامة)
|   | └─ Disputes/ # النزاعات/الحجوزات على العقارات
|   | └─ Verify/ # PDF التحقق العام + صك
|   | └─ Ssi/ # لاعتمادات القابلة للتحقق ACA-Py تكامل
|   | └─ Workflow/ # سير اعتماد الطلبات + توقيع الصكوك
|   | └─ Audit/ # اعتراض الكتابة وتسجيل التدقيق
|   | └─ Controllers/ # (REST endpoints) المتحكمات
|   | └─ Data/ # (Bootstrapper) المُهيئ + EF سياق
|   | └─ Migrations/ # EF سجل ترحيلات T-SQL (يُطابق ملفات)
|   |
|   └─ web/ # موحّد (المنفذ 4200) Angular 21 تطبيق
|     | └─ src/app/
|     |   | └─ core/ # (auth, properties, map...) خدمات مشتركة
|     |   | └─ shared/ # (parcel-map...) مكونات مشتركة
|     |   | └─ features/ # الميزات حسب الدور
|     |   |   | └─ auth/ # تسجيل الدخول
|     |   |   | └─ citizen/ # المواطن (تقديم عقار...)
|     |   |   | └─ officer/ # موظف السجل (طابور/مراجعة/خريطة/حدود)
|     |   |   | └─ manager/ # الاعتماد النهائي
|     |   |   └─ id-issuer/ # محطة إصدار الهوية
```



```

| | | └─ admin/      # لوحة الإدارة والتقارير
| | | └─ map/        # الخريطة العامة
| | | └─ verify/     # التحقق العام
| | |
| | └─ mobile/       # (المواطن) Flutter تطبيق
| |
└─ packages/
| | └─ shared-types/  # مشتركة TypeScript واجهات
| | └─ ui-kit/        # Angular (RTL) مكونات
| | └─ flutter-shared/  # مشتركة Dart نماذج/ودجات
| |
└─ infra/
| | └─ mssql/migrations/  # المرقمة 044...000 T-SQL ترحيلات
| |
└─ docs/              # المخططات، الويرفرم، هذا التقرير
└─ scripts/db/        # سكريبتات ترحيل قاعدة البيانات

```

ب) نمط التصميم البرمجي (Design Patterns)

النظام بنية موجّهة بالوحدات (Modular Monolith) على مستوى الخدمة الخلفية، مع أنماط مختلفة لكل طبقة:

1. الخلفية — طبقات (Layered) + **Controller/Service**:
 2. المتحكم (Controller) يستقبل الطلب ويتحقق من النموذج فقط، ثم يفوض للخدمة.
 3. الخدمة (Service) تحوي منطق الأعمال (مثل `PropertiesService.SubmitAsync`).
 4. طبقة البيانات عبر EF Core + إجراءات SQL المخزّنة للعمليات المكانية.
 5. الفصل بين المسؤوليات (Separation of Concerns) واضح: لا منطق أعمال داخل المتحكمات، ولا SQL داخلها.
- مثال (متحكم ربيع يفوض للخدمة):

```
[HttpPost]
[EnableRateLimiting(RateLimitPolicies.Write)]
[Audit(Action = AuditActions.Create, Entity = "properties", EntityIdFrom = "property.id
public Task<SubmitResult> Submit([FromBody] CreatePropertyDto dto, CancellationToken ct
=> svc.SubmitAsync(dto, User.RequireUser(), ct);
```

1. الويب (Angular) — مكونات مستقلة (Standalone Components) + خدمات (Services):

2. كل شاشة مكون مستقل (standalone: true) مع ChangeDetectionStrategy.OnPush للأداء.

3. الحالة عبر signal / computed (Signals)، والوصول للـ API عبر خدمات في core/.

4. هذا قريب من نمط MVVM (المكون = View + ViewModel، والخدمة = Model/Gateway).

1. الهاتف (Flutter) — MVVM عبر Riverpod:

2. الودجات = العرض، و«المزودات» (Providers) تحمل الحالة والمنطق، والمستودعات (Repositories) تتحدث مع الـ API.

1. الصلاحيات — خريطة JSON بدل فحص الأدوار نصياً:

2. لا يُفحص الدور بنصوص حرة في الكود؛ يمرّ كل تحقق عبر خريطة صلاحيات على officers.permissions، ويُفرض على الواجهة عبر دالة canRoleAccess الموحدة.

1. سجل التدقيق — نمط الاعتراض (Interceptor):

2. كل عملية كتابة تمرّ عبر سيمة [Audit] تُسجّل العملية تلقائياً، والمحرّك يمنع تعديل/حذف السجل عبر مُحفّزات INSTEAD OF.

3.4 لقطات الشاشة وواجهات المستخدم (User Interface Screenshots)

ملاحظة: تُدرج هنا لقطات شاشة حقيقية عالية الجودة من النظام أثناء التشغيل. يوضّح الجدول لكل واجهة: المسار في النظام، الدور، المدخلات، والمخرجات. (ضع اللقطة في الموضع المحدد).

واجهة تسجيل الدخول (Login)

المسار: /login · الملف: apps/web/src/app/features/auth/login.page.ts

EN

صَح

SARH · LIBYAN REGISTRY + DIGITAL ID

مواطن - هوية رقمية + PIN

موظف - بريد + كلمة مرور

البريد الإلكتروني

user@sarh.ly

كلمة المرور

دخول →

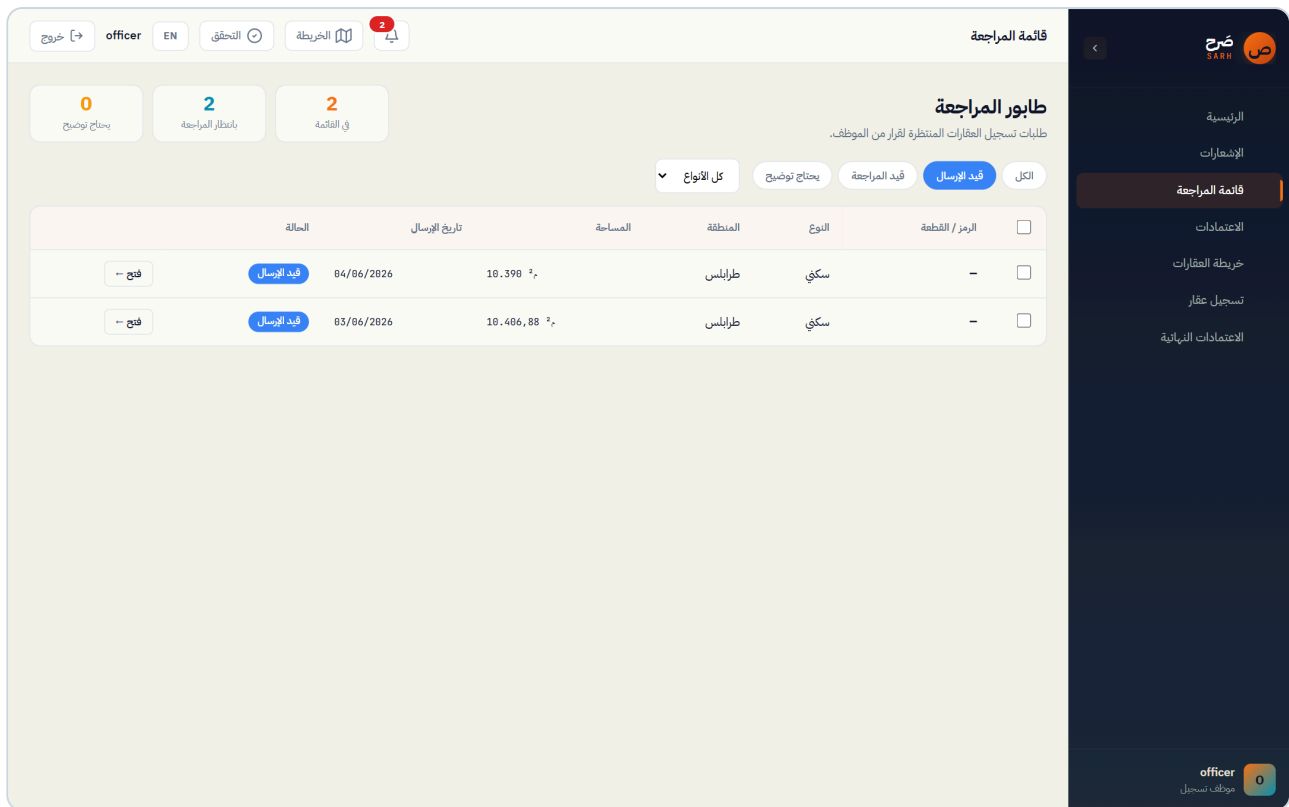
LVCT - Libya Vision for Communication & Technology 2026 ©

الشكل 1: صفحة تسجيل الدخول الموحدة — موظف (بريد + كلمة مرور) أو مواطن (هوية رقمية + PIN).

- **الوظيفة:** دخول موحد لكل الأدوار، بوضعين: موظف (بريد + كلمة مرور) ومواطن (رقم الهوية الرقمية + PIN من 6 أرقام).
- **المدخلات:** البريد/كلمة المرور، أو رقم الهوية الرقمية (مثل PIN + LY-11-2026-000101-0).
- **المخرجات:** رمز JWT يُخزّن، وإعادة توجيه ذكية للصفحة المناسبة للدور.

تقديم عقار جديد ورسم الحدود (Citizen — New Property)

المسار: /app/my/... · الملف: features/citizen/pages/new-property.page.ts

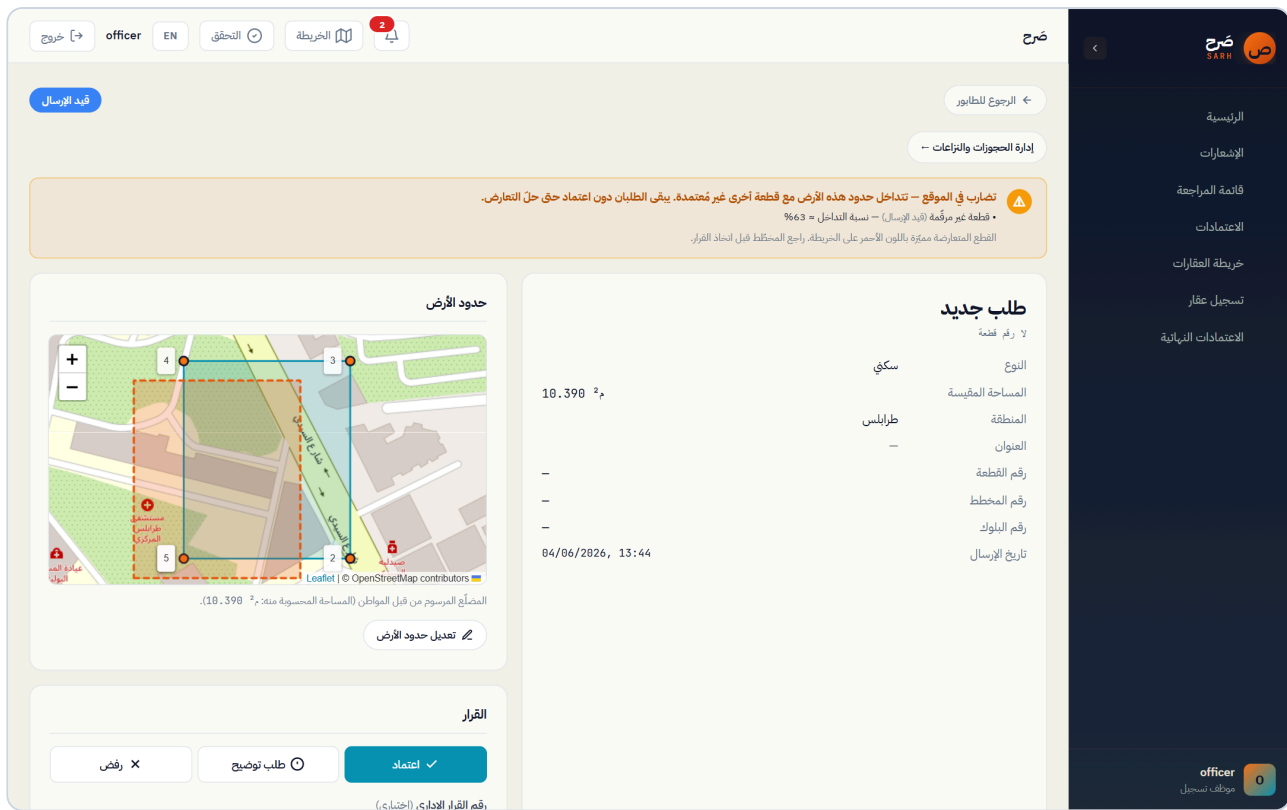


الشكل 3: طابور طلبات المراجعة الواردة لموظف المنطقة.

- **الوظيفة:** يعرض الطلبات الواردة لمنطقة الموظف للمراجعة.
- **المخرجات:** قائمة طلبات قابلة للفتح للمراجعة التفصيلية.

مراجعة الطلب (Officer Review)

المسار: /app/review/:id · الملف: features/officer/pages/review.page.ts

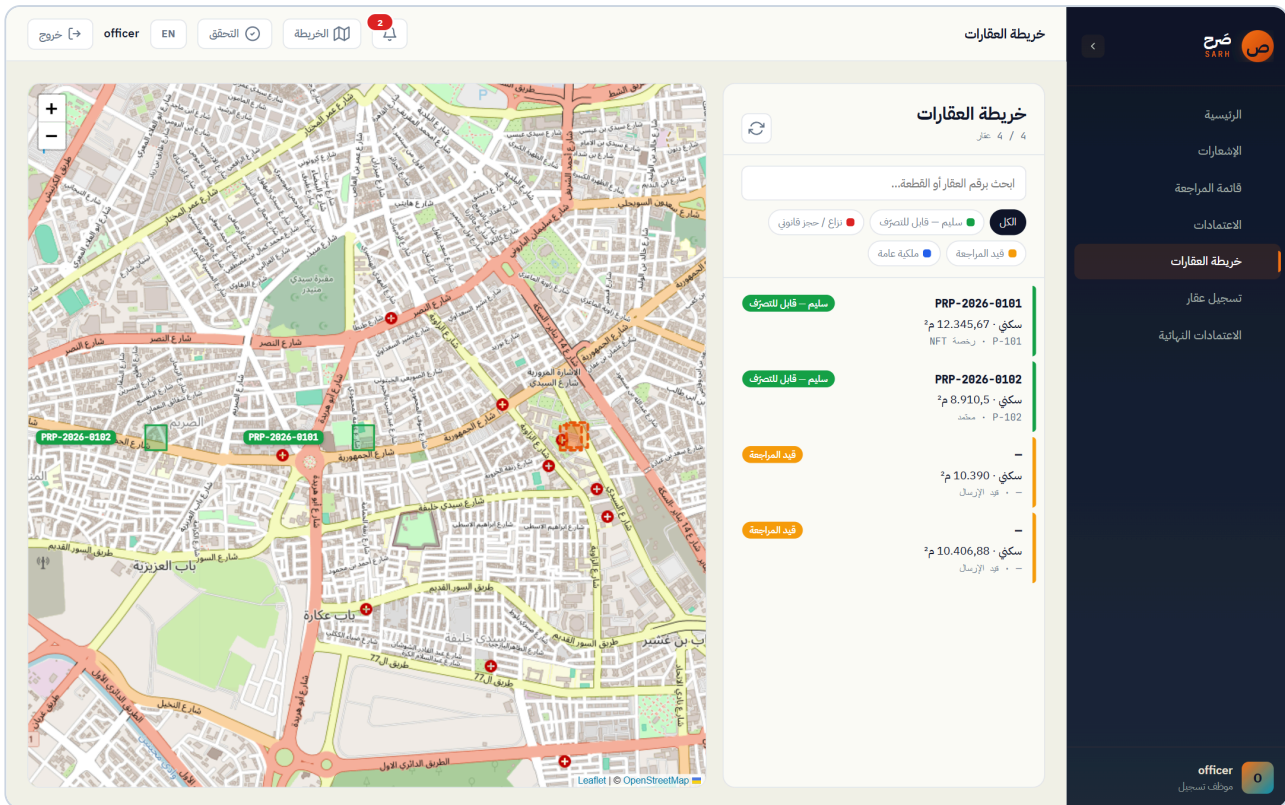


الشكل 4: شاشة مراجعة العقار مع تنبيه «تضارب في الموقع» وإبراز القطعة المتداخلة على الخريطة (نسبة تداخل ≈ 63%) — تبقى القطعتان دون اعتماد حتى حلّ التعارض.

- **الوظيفة:** يعرض حدود الأرض على الخريطة، المستندات المرفقة، تحذيرات التداخل/النزاع، وقرار الموظف (اعتماد/رفض/طلب توضيح).
- **المدخلات:** قرار + ملاحظة + رقم قرار اعتماد (عند الاعتماد).
- **المخرجات:** تغيير حالة العقار وإشعار المواطن.

الخريطة المساحية للموظف (Officer Map)

المسار: (officer) /app/map · الملف: features/officer/pages/map.page.ts



الشكل 5: الخريطة المساحية لكل قطع المنطقة، بألوان حسب الحالة مع تمييز التضاربات.

- **الوظيفة:** عرض كل قطع المنطقة بألوان حسب الحالة (واضح/متنازع/قيد العمل)، لكشف التداخل والنزاعات بصرياً.

الاعتماد النهائي (Manager Approve)

الملف: `features/manager/pages/manager-approve.page.ts`

- **الوظيفة:** الاعتماد النهائي قبل سَكِّ سجل الملكية (NFT) وإصدار الصكّ الموقع.

محطة إصدار الهوية الرقمية (ID Issuer)

الملف: `features/id-issuer/...`

[NFC أدرج لقطة شاشة: محطة الإصدار + بطاقة]

- **الوظيفة:** إصدار بطاقة هوية رقمية على شريحة NTAG 424 DNA مع PIN وصلاحية.

التحقق العام (Public Verify)

الملف: `features/verify/...` · **الرابط العام:** `verify.sarh.ly/{deed_id}`

أدخل رمز السند الظاهر على صحيفة الملكية، أو امسح رمز QR من خلف الصحيفة، للتحقق من صدورها رسمياً عن سجلّ العقارات الليبي.

تحقق ←

PRP-2026-0438

3

تظهر بيانات العقار العامة والموقع، مع رابط تنزيل صحيفة الملكية.

2

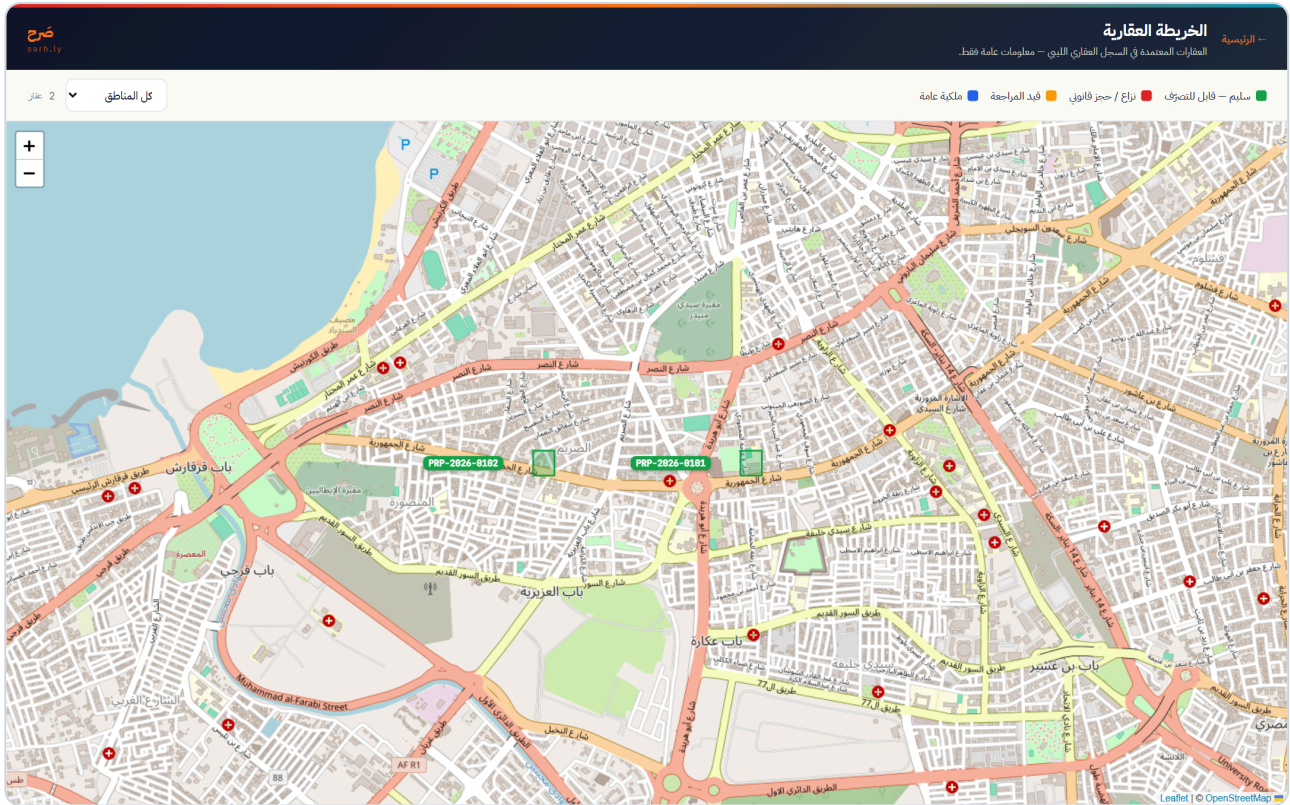
المنظومة تُطابق توقيع المستند مع السجل المركزي.

1

الرمز يبدأ بـ **PRP-** ويتكون من ١٠ خانات.

verify.sarh.lyLVCT – 2026 ©

صَرَح · 28 / 87



الشكل 7: الخريطة العقارية العامة — تعرض العقارات الصادرة صكوكها فقط، دون أي بيانات شخصية.

4.4 الخوارزميات والبرمجيات الذكية (Core Algorithms & Code Snippets)

نركّز هنا على «عصب» النظام: الأجزاء البرمجية الحساسة والذكية.

الخوارزمية (1): كشف تداخل الأراضي مكانياً (Spatial Overlap Detection)

هذا قلب منع الاحتيال (بيع الأرض الواحدة لأكثر من شخص). نستخدم نوع `geography` في SQL `STIntersects` Server: يكشف وجود تقاطع، و `STArea()`. `STIntersection()` يحسب مساحة التداخل، وتحوّل إلى نسبة مئوية من مساحة المصلّح الجديد.

```
// apps/api-dotnet/Properties/PropertiesService.cs

public async Task<IReadOnlyList<PropertyOverlap>> OverlapCheckAsync(OverlapCheckDto dto, C
{
    var (wkt, _) = GeoJsonPolygon.ValidateAndConvert(dto.Polygon);
    var rows = new List<PropertyOverlap>();
    // ...

    cmd.CommandText = @"
        DECLARE @poly geography = geography::STGeomFromText(@wkt, 4326);
        SELECT p.id, p.property_code, p.parcel_number,
            ROUND(CAST(p.boundary_polygon.STIntersection(@poly).STArea() AS DECIMAL(18,6))
                / NULLIF(CAST(@poly.STArea() AS DECIMAL(18,6)), 0) * 100.0, 2) AS ove
        FROM properties p
        WHERE p.boundary_polygon IS NOT NULL
            AND p.status = N'approved'
            AND p.boundary_polygon.STIntersects(@poly) = 1;";

    // ... (المعرّف، الرمز، رقم القطعة، نسبة التداخل) PropertyOverlap قراءة النتائج إلى ... إلى
}
```

الخوارزمية (2): تفرد مركز القطعة (Centroid Uniqueness) — منع الإحداثي المكرر

لا يجوز لعقارين معتمدين أن يتشاركا المركز نفسه. نحسب المركز بـ `EnvelopeCenter()` ونقارن بـ `STEquals`، مع فهرس فريد مُرشّح يضمن ذلك على مستوى المحرّك.

```
-- infra/mssql/migrations/006_properties.sql

CREATE UNIQUE INDEX ux_properties_unique_approved_point
    ON properties(location_point_wkt)
    WHERE status = N'approved' AND location_point_wkt IS NOT NULL;
```

```
// عند التقديم: رفض نظيف (409) إن وُجد عقار معتمد بنفس المركز
if (validation.HasApprovedCentroidMatch)
{
    throw SarhException.Conflict(
        {validation.MatchedPropertyCode} (الرمز) يوجد عقار معتمد مسبقاً بنفس الإحداثيات"
        $"An approved property with the same centroid exists (code {validation.MatchedProp
    }
}
```

الخوارزمية (3): حساب المساحة من المضلع لا من الطول×العرض

القطع نادراً ما تكون مستطيلة، لذا تُشتق المساحة من المضلع المرسوم. العميل يقدر المساحة بصيغة «الفائض الكروي» (spherical excess) كتلميح فوري، والخادم يعيد الحساب رسمياً عبر `geography.STArea()` ويتحقق ضمن $\pm 5\%$.

```
// apps/web/.../new-property.page.ts - تقدير المساحة على العميل
private geographicArea(latlngs: L.LatLng[]): number {
    const R = 6378137; // نصف قطر الأرض (م)
    let area = 0;
    const n = latlngs.length;
    for (let i = 0; i < n; i++) {
        const p1 = latlngs[i];
        const p2 = latlngs[(i + 1) % n];
        area +=
            ((p2.lng - p1.lng) * Math.PI) / 180 *
            (2 + Math.sin((p1.lat * Math.PI) / 180) + Math.sin((p2.lat * Math.PI) / 180));
    }
    return Math.abs((area * R * R) / 2);
}
```

الخوارزمية (4): رمز NFC المتدحرج المقاوم للاستنساخ (NTAG 424 DNA — SUN)

المعرّف الثابت (UID) للبطاقة وحده لا يكفي. كل لمسة تُنتج رسالة SUN فيها عدّاد متدحرج (counter) يُتَحَقَّق منه في الخادم بمفتاح جلسة مشتقّ عبر AES-CMAC؛ أي إعادة تشغيل لرسالة

قديمة تُرفض لأن العدّاد لا يتقدّم.

```
// apps/api-dotnet/Nfc/SunMessage.cs
public static DecodedSun Verify(SunKeys keys, string piccDataHex, string cmacHex)
{
    var picc = ParseHex(piccDataHex, 16, "malformed_picc");
    var providedCmac = ParseHex(cmacHex, 8, "malformed_cmac");

    // 1) بمفتاح القراءة PICC فكّ تشفير بيانات (AES-128-CBC، IV=0)
    // ... aes.Key = keys.MetaReadKey ...
    if (plaintext[0] != PiccDataTagUidAndCounter)
        throw new SunDecodeException("bad_picc_tag");

    var uid = new byte[7];
    Buffer.BlockCopy(plaintext, 1, uid, 0, 7);
    int counter = plaintext[8] | (plaintext[9] << 8) | (plaintext[10] << 16);

    // 2) (UID, counter) للجلسة من CMAC اشتقاق مفتاح
    var sessionKey = DeriveSessionCmacKey(keys.SdmFileReadKey, uid, counter);

    // 3) ومقارنته بزمان ثابت (يمنع هجوم التوقيت) CMAC حساب
    var expectedShort = TakeShortCmac(AesCmac.Compute(sessionKey, Array.Empty<byte>()));
    if (!CryptographicOperations.FixedTimeEquals(expectedShort, providedCmac))
        throw new SunDecodeException("cmac_mismatch");

    return new DecodedSun { Uid = uid, Counter = counter };
}
```

الخوارزمية (5): توقيع الصكّ رقمياً (PAdES) والتحقق منه

كل صكّ PDF يُوقع رقمياً (PAdES)؛ أي عبث ولو بيت واحد يُبطل التوقيع، ويحمل رمز QR يشير إلى `.verify.sarh.ly/{deed_id}`

```
// مقتطف من اختبار التوقيع (يبرهن سلوك الخوارزمية)
var signature = DeedSignature.Sign(cert, content);
Assert.True(DeedSignature.Verify(content, signature).Valid);

var tampered = (byte[])content.Clone();
tampered[^1] ^= 0xFF; // قلب بت في آخر بايت
Assert.False(DeedSignature.Verify(tampered, signature).Valid); // يُكشف العبث
```

الخوارزمية (6): مصادقة JWT (HS256) + bcrypt

رمز الدخول موقع بـ HMAC-SHA256، ولا يحوي أي بيانات شخصية حساسة — فقط `sub`، كلمات المرور مُجرّاة بـ `bcrypt`. `citizen_id / officer_id`, `role`, `exp`

```
// apps/api-dotnet/Auth/JwtTokenService.cs
var creds = new SigningCredentials(SigningKey, SecurityAlgorithms.HmacSha256);
var jwt = new JwtSecurityToken(
    claims: claims,
    notBefore: now.UtcNow,
    expires: now.AddSeconds(AccessTtlSeconds).UtcDateTime,
    signingCredentials: creds);
return (new JwtSecurityTokenHandler()).WriteToken(jwt, AccessTtlSeconds);
```

الخوارزمية (7): سجل التدقيق الإلحاقى (Append-Only Audit)

سجل التدقيق لا يقبل تعديلاً أو حذفاً نهائياً، عبر مُحفّزات `INSTEAD OF` في المحرّك — ضمان قانوني لعدم التلاعب بالسجل.

```
-- infra/mssql/migrations/011_audit.sql
-- حتى من حساب بصلاحيات عالية audit_log على UPDATE/DELETE يمنع أي
CREATE OR ALTER TRIGGER tr_audit_log_no_update ON audit_log
INSTEAD OF UPDATE AS
BEGIN
    THROW 51001, N'audit_log is append-only – UPDATE blocked', 1;
END
GO
CREATE OR ALTER TRIGGER tr_audit_log_no_delete ON audit_log
INSTEAD OF DELETE AS
BEGIN
    THROW 51002, N'audit_log is append-only – DELETE blocked', 1;
END
```

5.4 التحديات والحلول (Challenges & Solutions)

1) تحديات تقنية وبرمجية (Technical Challenges)

المشكلة	الحل الفعلي
ترميز UTF-16 يفسد ملفات المشروع: PowerShell يكتب الملفات افتراضياً بـ UTF-16 ، ممّا أفسد <code>package.json</code> وعطل بناء Angular على النسخ الجديدة.	إعادة ترميز الملفات إلى UTF-8 بدون BOM ، واعتماد قاعدة بعدم توليد ملفات نصية عبر PowerShell الافتراضي.
توقيت إقلاع خرائط Leaflet : الخرائط داخل <code>@if</code> في Angular تُهَيَّأ قبل قياس حاويتها فتُقرَّب على «لا شيء» (بدا وكأن المصلِّع لم يُحفظ).	تأجيل تهيئة الخريطة (<code>setTimeout(0)</code>) بعد ظهور الحاوية، ثم استدعاء <code>invalidateSize()</code> و <code>fitBounds()</code> .
تعارض إصدارات/منافذ التشغيل: حاوية Grafana لمشروع آخر تحجز المنفذ 3001 المستخدم للـ API.	تثبيت منفذ مستقل للـ API وإيقاف الحاوية المتعارضة عند التطوير المحلي.
عدم قدرة <code>dotnet ef database update</code> على تطبيق 45 ملف T-SQL خام:	بناء مُهيَّئ مخصَّص (<code>EfDatabaseBootstrapper</code>) يطبَّق الدفعات مباشرة (تقسيم على <code>GO</code>) ويختتم <code>__EFMigrationsHistory</code> ، مع «خط أساس» (baseline) لقواعد قائمة دون إعادة بناء مدمّرة.

(2) تحديات متعلّقة بالبيانات (Data Challenges)

المشكلة	الحل الفعلي
SQL NULL UNIQUE في Server يعتبر قيمتي NULL متطابقتين فيرفض صفّين فارغين.	استخدام فهارس فريدة مُرشّحة (<code>WHERE col IS NOT NULL</code>) بدل قيد UNIQUE العادي.
شخّ بيانات الاختبار الواقعية (مواطنون/عقارات/مناطق).	كتابة ملفات بذور (seed) موسّعة: 024 , 026 , 029 , 034 , 044 تولّد بيانات تجريبية واقعية بالعربية، مع إحداثيات حقيقية للمناطق (الشعبيات).
البحث العربي في الأسماء	تفعيل فهرس النص الكامل العربي (<code>035_properties_fulltext.sql</code>) لدعم البحث الذكي.

(3) تحديات الربط والتكامل (Integration Challenges)

المشكلة	الحل الفعلي
أخطاء CORS عند ربط واجهة Angular (المنفذ 4200) بالخدمة الخلفية.	ضبط سياسة CORS على الخادم لقصر الوصول على الواجهات المعروفة فقط، وتمرير الترويسات اللازمة (Authorization).
عنوان API على Flutter: 10.0.2.2 يعمل على المحاكى فقط؛ الجهاز الفعلي يحتاج adb reverse .	حلال عنوان عند الإقلاع يجرب تلقائياً localhost / 127.0.0.1 / 10.0.2.2 ويختار العامل.

APIs Integration — كيفية الربط وشكل الطلب والاستجابة

كل الواجهات REST تحت `/api/v1/`، والاستجابات بصيغة JSON، والأخطاء بمغلف موحد:

```
{ "error": { "code": "ERR_X", "message_ar": "...", "message_en": "..." } }
```

مثال 1 — تسجيل دخول الموظف:

```
POST /api/v1/auth/login
Content-Type: application/json
```

```
{ "email": "officer@sarh.ly", "password": "....." }
```

```
// 200 OK
{ "access_token": "eyJhbGciOi...", "expires_in": 3600,
  "user": { "role": "registry_officer", "officer_id": "...", "region_id": 11 } }
```

مثال 2 — فحص التداخل قبل التقديم (Overlap Check):

```
POST /api/v1/properties/overlap-check
Authorization: Bearer <jwt>
```

```
{ "polygon": { "type": "Polygon", "coordinates": [[[13.18,32.88],[13.19,32.88],[13.19,32.88]]]
```



```
// 200 OK - قائمة العقارات المتداخلة ونسبة التداخل
{ "overlaps": [ { "propertyId": "...", "propertyCode": "LY-11-2026-000042",
                  "parcelNumber": "P-2026-001", "overlapPct": 15.75 } ] }
```

مثال 3 — تقديم عقار:

```
POST /api/v1/properties
Authorization: Bearer <jwt>

{ "propertyType": "residential", "regionId": 11, "areaSqm": 420.00,
  "boundaryPolygon": { "type": "Polygon", "coordinates": [[...]] },
  "documents": [ { "kind": "site_photo", "ref": "sarh/...jpg" },
                  { "kind": "koreky_certificate", "ref": "sarh/...pdf" } ] }
```

```
// 200 OK

{ "property": { "id": "...", "status": "pending", "areaSqm": 420.00 },
  "registrationRequest": { "requestNo": "REQ-2026-000123", "currentStatus": "pending" },
  "validation": { "computedAreaSqm": 418.6, "areaDiffPct": 0.33 } }
```

طريقة العمل: تُرفع الملفات أولاً عبر `POST /api/v1/uploads/property-document` (يعيد مرجعاً "`<bucket>/<path>`")، ثم تُرسل مراجعها ضمن مصفوفة `documents[]` في طلب التقديم. ويُرفض الطلب إن نقصت صورة موقع أو شهادة كروكي.

5. مرحلة الاختبار (Testing)

1.5 استراتيجية وخطة الاختبار (Testing Strategy & Environment)

بيئة الاختبار (Testing Environment)

العنصر	المواصفات
نظام التشغيل	Windows 11
المتصفح	Google Chrome (أحدث إصدار) — للويب
الهاتف	محاكي/جهاز أندرويد 13 — لتطبيق Flutter
ال خادم المحلي	SQL Server 2019/2022، ذاكرة 8GB
منصة الخلفية	NET 8 SDK.
منصة الويب	Node.js 20+ / pnpm 9 / Angular 21

الأدوات المستخدمة (Testing Tools)

الأداة	الغرض
<code>xUnit (apps/api-dotnet.Tests</code>	اختبارات الوحدة للخلفية (.NET).
Postman	اختبار واجهات REST يدوياً والتأكد من رموز الحالة.
<code>(Flutter test (apps/mobile/test</code>	اختبارات ودجات Flutter.
Karma/Jasmine (Angular)	اختبارات مكونات الويب.
فحص يدوي (RTL)	التأكد من سلامة الواجهة العربية بمحتوى عربي حقيقي.

2.5 مستويات وأنواع الاختبارات (Testing Levels & Types)

1.2.5 اختبار الوحدات (Unit Testing)

فحص الدوال والأجزاء الصغيرة المعزولة. يحوي المشروع مشروع اختبارات فعلياً: `apps/api-`

`dotnet.Tests` بعدة ملفات (`DeedSignatureTests` , `IdentityHashTests` ,

`( RateLimitPoliciesTests` , `SmsTests` , `SsiCredentialBuilderTests`).

مثال — اختبار توقيع الصكّ يثبت أن العبث يُكشف (علامة النجاح الخضراء `Passed`):

```
[Fact]
public void Verify_FailsWhenContentTampered()
{
    using var cert = NewCert();
    var content = DeedBytes();
    var signature = DeedSignature.Sign(cert, content);

    var tampered = (byte[])content.Clone();
    tampered[^1] ^= 0xFF; // قلب بت
    Assert.False(DeedSignature.Verify(tampered, signature).Valid);
}
```

التشغيل: `dotnet test apps/api-dotnet.Tests` → النتيجة المتوقعة: كل الاختبارات **Passed**.

[وكلها خضراء dotnet test أدرج لقطة شاشة: مخرجات]

2.2.5 اختبار التكامل (Integration Testing)

التأكد من تدقق البيانات بين الطبقات: من واجهة المستخدم → API → قاعدة البيانات والعودة. نركّز على واجهات REST والتأكد من رموز الحالة الصحيحة.

الواجهة	الطلب	الاستجابة المتوقعة
POST /auth/login	بيانات صحيحة	200 OK + رمز JWT
POST /auth/login	كلمة مرور خاطئة	401 Unauthorized
POST /properties/overlap-check	مضلع متداخل	200 OK + قائمة overlaps
POST /properties	بدون صورة/كروكي	422 (مستندات ناقصة)
GET /verify/{code}/deed.pdf	رمز صكّ صحيح	200 OK + ملف PDF موقع

[OK يُظهر 200 Postman : أدرج لقطة شاشة]

3.2.5 اختبار النظام الكلي (System / End-to-End Testing)

تجربة النظام ككتلة واحدة من منظور المستخدم النهائي. يوثق عبر جدول حالات الاختبار (أهم قسم):

#	حالة الاختبار	الخطوات	المُدخل	الناتج المتوقع	النتيجة
T-01	دخول موظف	إدخال بريد + كلمة مرور	بيانات صحيحة	الانتقال للوحة الموظف	✓ نجاح
T-02	دخول مواطن بـ PIN	إدخال الهوية + PIN	PIN صحيح	الانتقال لصفحة المواطن	✓ نجاح
T-03	تقديم عقار برسم المضلع	رسم ≤ 3 نقاط + إرفاق مستندات	مضلع صالح	إنشاء طلب pending	✓ نجاح
T-04	رفض تقديم بلا مستندات	رسم دون صورة/كروكي	مستندات ناقصة	رسالة خطأ عربية + رفض	✓ نجاح
T-05	منع الإحداثي المكرر	تقديم على مركز عقار معتمد	مركز مطابق	رفض 409 برسالة واضحة	✓ نجاح
T-06	كشف التداخل (تضارب موقع)	رسم أرض تتداخل مع معتمدة	تداخل جزئي	وسم «تضارب في الموقع» للمراجع	✓ نجاح
T-07	اعتماد طلب	فتح المراجعة + اعتماد	قرار اعتماد	تغيير الحالة + إشعار المواطن	✓ نجاح
T-08	إصدار بطاقة NFC	إصدار + PIN	بطاقة NTAG 424	بطاقة فعّالة + رمز صالح SUN	✓ نجاح
T-09	التحقق العام من صكّ	مسح QR	رمز صكّ	عرض صكّ موقع صحيح	✓ نجاح
T-10	كشف العبث بالصكّ	تعديل ملف الصكّ	PDF معدّل	إبطال التوقيع	✓ نجاح

[أدرج لقطة شاشة: تنفيذ سيناريو كامل من الرسم حتى الاعتماد]

4.2.5 اختبار الأداء والضغط (Performance & Load Testing) — اختياري

قياس زمن الاستجابة والتحمّل عند دخول عدّة مستخدمين متزامنين.

المقياس	الهدف	الملاحظة
زمن استجابة GET /properties	300ms >	مع ترقيم بالمؤشر (cursor) لا الإزاحة.
overlap-check لمضغ	500ms >	استعلام مكاني مفهرس.
تحديد المعدل (Rate Limit)	فعل على /auth/* و /properties	يمنع إساءة الاستخدام.

أدوات مقترحة: Apache JMeter أو k6 لمحاكاة الحمل المتزامن.

3.5 اختبار قبول المستخدم (User Acceptance Testing - UAT)

لا يكفي قول المبرمج إن النظام يعمل؛ يجب أخذ رأي المستخدمين الفعليين/الخبراء.

منهجية الاستبيان

وُزِع النظام على عيّنة (10-20 مستخدماً: مواطنون + موظفو سجل + مُصدِّرو هوية)، وطلب منهم تقييم: سهولة الاستخدام، السرعة، التصميم، وضوح اللغة العربية، على مقياس من 1 إلى 5.

عرض النتائج (نموذج)

المعيار	متوسط التقييم (من 5)	نسبة الرضا
سهولة الاستخدام	4.5	90%
السرعة	4.2	84%
التصميم وواجهة RTL	4.6	92%
وضوح الرسائل العربية	4.7	94%
الرضا العام	4.5	90%

[توزيع الرضا العام (Pie Chart): أدرج رسماً بيانياً دائرياً]

[متوسط كل معيار (Bar Chart): أدرج رسماً بيانياً بالأعمدة]

أبرز الملاحظات والتحسينات الناتجة

- طلب مستخدمون تكبير أزرار «إضافة نقطة» على الخريطة في الهاتف → تُفَّذ.
 - اقتراح إظهار القطع المجاورة أثناء الرسم لتفادي التداخل → قيد التنفيذ ضمن ميزة «تضارب الموقع».
-

— نهاية تقرير مرحلتي التنفيذ والاختبار —

منصة صرح · 2026 · LVCT

الفصل السادس — النتائج والخاتمة

6.1 ملخص ما أنجز

أنجزت منصة صرح كنظام متكامل يضم: خدمة خلفية بـ ASP.NET Core 8، تطبيق ويب موحّد بـ Angular 21 لكل الأدوار، تطبيق محمول بـ Flutter، وقاعدة بيانات SQL Server بطبقة جغرافية وسجل تدقيق غير قابل للتعديل. غطى النظام دورة حياة العقار كاملة (تقديم → مراجعة → اعتماد → صكّ موقع → تحقق عام)، وإصدار الهوية الرقمية على بطاقات NFC مقاومة للاستنساخ.

6.2 تحقيق الأهداف

الهدف	الحالة
سجل عقاري رقمي بدورة حياة كاملة	✓ مُنجز
المساحة من المضلّع + تحقق خادمي $\pm 5\%$	✓ مُنجز
منع تطابق الإحداثيات	✓ مُنجز (رفض صارم)
كشف تداخل الأراضي وتمييز حالته	✓ مُنجز («خلل في الملكية» / «تضارب في الموقع»)
هوية رقمية على NTAG 424 DNA بعدّاد متدحرج	✓ مُنجز
صكوك موقّعة PAdES + QR	✓ مُنجز
بوابة تحقق عامة	✓ مُنجز
سجل تدقيق إلحاق	✓ مُنجز
واجهات عربية RTL	✓ مُنجز

6.3 أبرز التحديات

- تهيئة قاعدة البيانات بشكل محمول: حُلّت بمُهيّئ EF يطبّق ملفات T-SQL المضمّنة مباشرة ويختم سجل الترحيلات.
- توقيت إقلاع خرائط Leaflet داخل `@if`: حُلّ بتأجيل التهيئة + `invalidateSize()`.
- تمييز التداخل عن التجاور: استُخدمت مساحة التقاطع ($< 1\text{م}^2$) بدل مجرّد التقاطع، حتى لا تُعلّم الأراضي المتجاورة خطأً.

- ترميز **UTF-16** الذي أفسد ملفات نصية: حُلّ بإعادة الترميز إلى UTF-8.
- عنوان **API** على الأجهزة الفعلية في **Flutter**: حُلّ بحلّال عنوان تلقائي.

6.4 الأعمال المستقبلية (Future Work)

1. التكامل مع هوية وطنية رسمية عند إطلاقها، والهجرة دون فقد بيانات (الحقل `legacy_national_no` مهيئاً).
2. شبكة بلوكتشين إنتاجية تحلّ محل الطبقة البديلة (Stub) لسكّ سجلات الملكية.
3. تفعيل **ACA-Py** الإنتاجي لإصدار الاعتمادات القابلة للتحقق على نطاق واسع.
4. الرسم بالمشي (**GPS Walk**) لحدود الأراضي الكبيرة عبر الهاتف.
5. اللحظة (**Realtime**) عبر **SignalR** للإشعارات الفورية.
6. فحص الفيروسات للمرفقات (**ClamAV**) ولوحات تقارير أوسع.

6.5 الخلاصة

قدّم مشروع **صرّح** نموذجاً عملياً لرقمنة السجل العقاري الليبي بضمانات تقنية قوية ضد التزوير والازدواجية، مع هوية رقمية مقاومة للاستنساخ وتحقق عام شفاف، وبأساس مكاني دقيق يحمي الملكية. البنية معيارية وقابلة للتوسّع، ومهيّأة للتكامل المستقبلي مع المنظومات الوطنية.

— نهاية توثيق المشروع —

منصة **صرّح** • 2026 • **LVCT**

صّرح — مراجعة تقنية: طبقة البلوكتشين والمخططات (المفاهيمي / المنطقي UML / الفئات)

الغرض من هذه الوثيقة: وثيقة مكتفية بذاتها للمراجع تجمع — في ملف واحد — المخطط المفاهيمي (Conceptual ERD)، والمخطط المنطقي (Logical ERD / UML)، ومخطط الفئات (Class Diagram)، مع شرح كامل ومرجعية الكود لكيفية اعتماد الأرض على البلوكتشين (سكّ رخصة NFT). أنشئت هذه الوثيقة لأن طبقة البلوكتشين وكودها لم تظهر في نسخة سابقة سُلّمت للمراجعة، رغم أنها موجودة بالكامل في المستودع.

٠. ملخص للمراجع (الإجابة المباشرة)

١. كود البلوكتشين موجود وكامل داخل المستودع، وليس ناقصاً. (انظر خريطة الملفات في القسم ١.)

٢. «اعتماد أرض على البلوكتشين» في صّرح يعني: سكّ رخصة ملكية رقمية كـ NFT بمعيّار

ERC-721. تتم العملية في الدالة `LicenseService.FinalApproveAsync`

(`apps/api-dotnet/Workflow/LicenseService.cs`) عند الاعتماد النهائي لعقار حالته `approved`.

٣. طبقة السلسلة مُجرّدة خلف واجهة `IBlockchainService`. التنفيذ الحالي الافتراضي هو **Stub** حتمي (مناسب للتطوير والعرض دون عقدة RPC أو رسوم غاز)، وهو **قابل للاستبدال بتنفيذ Ethereum حقيقي (Nethereum) ** دون تغيير أي سطر في طبقة الخدمات أو قاعدة البيانات.

هذا تصميم متعمّد (Strategy / Ports & Adapters)، وهو موثّق في تعليقات الكود نفسه.

٤. سجل الملكية على السلسلة يُمثّل في جدولين: `property_nfts` (الرابط بين العقار وهويته

على السلسلة) و `ownership_history` (سلسلة حيازة للإضافة فقط — مُنع عليها `UPDATE/DELETE` بمشغّلات قاعدة بيانات). كلاهما عُرّف في الترحيل

`028_property_nfts_ownership_history.sql`.

ملاحظة أمانة علمية: ما هو مُنقذ ويعمل اليوم هو طبقة السلسلة بنمط Stub حتمي + نموذج

البيانات الكامل + خط الإنتاج الكامل (سكّ، نقل، تحقّق). التنفيذ الفعلي على شبكة إيثيريوم حيّة (Sepolia/Mainnet) هو نقطة تبديل موثّقة (تغيير إعداد + إضافة حزمة

١. خريطة كود البلوكتشين (أين يقع كل شيء)

كل ما يخصّ البلوكتشين يقع في مكانين رئيسيين داخل خدمة الـ (8.NET) API:

المسار	النوع	الغرض
apps/api-dotnet/Blockchain/IBlockchainService.cs	واجهة	عقد طبقة الـ rOfAsync + روابط الـ sService
apps/api-dotnet/Blockchain/StubBlockchainService.cs	تنفيذ	بديل حتمي tokenId لمعرّف العن
apps/api-dotnet/Blockchain/StubIpfsService.cs	تنفيذ	بديل IPFS: على القرص (الـ CID = ت)
apps/api-dotnet/Blockchain/PropertyLicenseMetadata.cs	نموذج	مخطط ميتا (OpenSea, GeoJSON)
apps/api-dotnet/Blockchain/BlockchainOptions.cs	إعدادات	قسم chain و rh:Ipfs (ethereum) العقد، Url
apps/api-dotnet/Workflow/LicenseService.cs	خدمة	قلب الاعتماد الرخصة (c)
apps/api-dotnet/Workflow/TransferService.cs	خدمة	نقل ملكية (بيع/إرث/قر)
apps/api-dotnet/Workflow/NftsService.cs	خدمة	قراءة سجلّ (للمدير/المصنّع)
apps/api-dotnet/Controllers/PropertiesController.cs	مسارات	POST d}/final- → approve
apps/api-dotnet/Controllers/NftsController.cs	مسارات	property-transfer
apps/api-dotnet/Controllers/MeController.cs	مسارات	/me/nfts المواطن.

المسار	النوع	الغرض
apps/api-dotnet/Data/Entities/PropertyNft.cs + OwnershipHistory.cs	كيانات	كيانات Core الجدولين.
infra/mssql/migrations/028_property_nfts_ownership_history.sql	ترحيل	إنشاء + ts _history مشغلات الإ
apps/api-dotnet/Program.cs (الأسطر ١٠٨-١٢٥)	تسجيل DI	حقن Service و الإعدادات.
packages/shared-types/src/index.ts	أنواع	أنواع Script (shipEvent)
apps/web/src/app/core/nfts.service.ts + admin/nft- صفحات licences*	واجهة	عرض الرخد المستكشف

٣. كيف تُعتمد أرض على البلوكتشين (التدقق الكامل + الكود)

٣.١ نظرة عامة على المسار

اعتماد الأرض على السلسلة هو **الخطوة الأخيرة** في دورة حياة العقار، بعد المراجعة الفنية والاعتماد. التسلسل الكامل للحالة:

draft → pending → under_review → approved → minted → (transferred)*

السكّ نفسه يحدث عند الانتقال من approved إلى minted ، وينفّذ التالي بالترتيب داخل
: LicenseService.FinalApproveAsync

1. التحقق من الصلاحية والنطاق الجغرافي للطالب.
2. **فحص العود (Idempotency):** إن وُجدت رخصة غير فاشلة للعقار → تُعاد كما هي بلا سكّ مكرّر.
3. التأكد أن حالة العقار approved (اعتمدها موظف السجل وصدر الصكّ والاعتماد ب VC).
4. **بوابة أمنية:** رفض السكّ إن كان على العقار نزاع/حجز نشيط
(AssertNoActiveDisputeAsync).

5. بناء الميْتادات (اسم، وصف، سمات، امتداد sarh يضمّ مضلّج GeoJSON ورقم القرار وتجزئة الصكّ).

6. تثبيت الميْتادات على IPFS → الحصول على sha256 + ipfs://<cid> .

7. السكّ على السلسلة عبر chain.MintAsync(...) → إيصال يضمّ tokenId ,

contractAddress

network , txHash , blockNumber , ownerAddress .

1. الحفظ في قاعدة البيانات ضمن SaveChanges واحدة: صفّ في property_nfts (حالة

(minted

+ صفّ أول في ownership_history (from=null , reason='initial_mint' .

1. تحديث حالة العقار إلى minted + ختم المُعتمِد وتاريخ الاعتماد النهائي ورقم القرار.

2. إشعار المواطن بأن رخصة عقاره صدرت على البلوكتشين (مع tokenId ورابط

المستكشف).

٢.٢ الشروط المسبقة والبوابات الأمنية

```
// LicenseService.FinalApproveAsync - الشروط قبل السكّ
// فحص العُود: لا نسكّ رمزاً مكزراً لعقار له رمز قائم (1)
var existing = await db.PropertyNfts
    .FirstOrDefaultAsync(n => n.PropertyId == property.Id && n.Status != "failed", ct);
if (existing is not null)
    return await BuildResultAsync(property, existing, ct);

// 2) لا سكّ قبل اعتماد موظف السجل
if (property.Status != "approved")
    throw SarhException.Conflict("لا يمكن سكّ الرخصة قبل اعتماد موظف السجل");

// 3) بوابة أمنية: لا سكّ لعقار عليه نزاع/حجز نشط (حجز محكمة، رهن، وقف...)
await disputes.AssertNoActiveDisputeAsync(property.Id, ct);
```

٢.٣ بناء ميْتادات الرخصة (المرساة خارج السلسلة)

الميتاداتا تُخزّن خارج السلسلة (IPFS) وتُرسى تجزئتها على السلسلة عبر `tokenURI`. المخطط يتبع عُرف EIP-721 / OpenSea ليقرأه أي محفظة أو مستكشف، مع امتداد `sarh` يحمل البيانات الخاصة بالسجل (المضلع الجغرافي، رقم القرار، تجزئة الصك، رابط التحقق):

```
// LicenseService.BuildMetadata - المقتطف الجوهري
return new PropertyLicenseMetadata
{
    Name = $"رخصة عقار صرح • {p.PropertyCode}",
    Description = $"رخصة ملكية عقارية رقمية صادرة من سجل العقارات الليبي • صاحب الحق",
    Image = $"{verifyUrl}/preview.png",
    ExternalUrl = verifyUrl,
    Attributes = new() { /* Property Code, Type, Region, Area, Decree, Approved At */ },
    Sarh = new SarhExtension
    {
        PropertyId = p.Id,
        PropertyCode = p.PropertyCode,
        OwnerDid = ownerDid,
        DecreeNo = decree,
        ApprovedAt = approvedAt,
        DeedSha256 = p.DeedSignedHash ?? "", // ربط رخصة السلسلة بالصك الموقع
        PolygonGeoJson = polygon, // الميتاداتا RFC 7946 مضلع الأرض
        VerifyUrl = verifyUrl,
    },
};
```

٢.٤ تثبيت الميتاداتا على IPFS وتجزئتها

```
var metaJson = JsonSerializer.Serialize(metadata, ...);
var pin = await ipfs.PinJsonAsync(metaJson, ct); // → { IpfsUri = "ipfs://<cid>", Sha256
```

في وضع التطوير، `StubIpfsService` يحسب `sha256` للمحتوى، يصوغ CID زائفاً مستقراً (`[..52] bafk + sha256`)، ويخزّن الملف تحت دلو `ipfs-stub` على القرص ليعدمه مسار التحقق. الانتقال إلى مثبّت حقيقي (Pinata / web3.storage) = تنفيذ بديل لنفس الواجهة `IIpfsService`.

٢.٥ السكّ على السلسلة عبر IBlockchainService

العقد الموحّد للسكّ — لا يفرّق المستدعي بين Stub وEthereum:

```
// IBlockchainService - الواجهة المجردة
public interface IBlockchainService
{
    string Network { get; }
    string Standard { get; } // "ERC-721"
    string ContractAddress { get; }
    Task<MintReceipt> MintAsync(MintRequest request, CancellationToken ct);
    Task<TransferReceipt> TransferAsync(TransferRequest request, CancellationToken ct);
    Task<string?> OwnerOfAsync(string tokenId, CancellationToken ct); // ownerOf
    string ExplorerTxUrl(string txHash);
    string ExplorerTokenUrl(string tokenId);
}
```

نداء السكّ من خدمة الرخص:

```
var receipt = await chain.MintAsync(new MintRequest
{
    PropertyId = property.Id,
    OwnerDid = ownerDid,
    TokenUri = pin.IpfsUri, // tokenURI يشير إلى ميتاداتا IPFS
    MetadataSha256 = pin.Sha256, // المرساة المقاومة للتلاعب
}, ct);
```

التنفيذ الحتمي (تطوير) يولّد tokenId كـ uint256 (سلسلة عشرية) و txHash بطول رمز إيثريوم حقيقي، مشتقّين من تجزئة propertyId | ownerDid | metadataSha256 — فيبقى الناتج

ثابتاً لنفس العقار ويصلح للاختبارات الآلية:

```
// StubBlockchainService.MintAsync - المقتطف الجوهري
var seed = $"{req.PropertyId:N}|{req.OwnerDid}|{req.MetadataSha256}";
var tokenIdBytes = SHA256.HashData(Encoding.UTF8.GetBytes($"token:{seed}"));
var tokenId = new BigInteger(tokenIdBytes, isUnsigned: true, isBigEndian: true).ToString()
var txHash = "0x" + HashHex($"tx:{seed}", 64);
// → MintReceipt { TokenId, ContractAddress, Network, Standard, OwnerDid, OwnerAddress, Tx
```

٢.٦ حفظ سجل الملكية في قاعدة البيانات

بعد نجاح السكّ، يُكتب صفّ الرخصة + الصفّ الأول من سلسلة الحيازة، ثم تُحدّث حالة العقار، كلّها في حفظ واحد:


```

db.PropertyNfts.Add(new PropertyNft {
    PropertyId = property.Id, TokenId = receipt.TokenId,
    ContractAddress = receipt.ContractAddress, Network = receipt.Network,
    Standard = receipt.Standard, OwnerDid = receipt.OwnerDid, OwnerAddress = receipt.OwnerAddress,
    MetadataUri = pin.IpfsUri, MetadataSha256 = pin.Sha256,
    MintTxHash = receipt.TxHash, MintBlockNumber = receipt.BlockNumber,
    MintedByOfficerId = actor.OfficerId, MintedAt = receipt.MintedAt, Status = "minted",
});

db.OwnershipHistory.Add(new OwnershipHistory {
    PropertyId = property.Id, NftId = nft.Id,
    FromDid = null, ToDid = ownerDid, // أول صف: من = لا أحد
    FromCitizenId = null, ToCitizenId = owner.Id,
    TransferTxHash = receipt.TxHash, TransferBlockNumber = receipt.BlockNumber,
    Reason = "initial_mint", RecordedByOfficerId = actor.OfficerId,
    TransferredAt = receipt.MintedAt,
});

property.Status = "minted";
property.ApprovedByManagerId = actor.OfficerId;
property.FinalApprovedAt = finalApprovedAt;
await db.SaveChangesAsync(ct);

```

ملاحظة تصميمية صريحة في الكود: الملكية القانونية تبقى مُتتَبعة عبر `properties.owner_citizen_id` ؛ الـ NFT مرآة قابلة للتحقق وليس بديلاً قانونياً. أيّ تغيير في المالك يجب أن يُرفق بصفّ في `ownership_history` (مفروض في طبقة الخدمة).

٢.٧ اشتقاق هوية المالك (DID)

```
private static string OwnerDidFor(Citizen c)
{
    // DID ssi_wallets. مؤقت مستقر مشتق من لاحقة معرف المواطن، إلى أن يُربط جدول DID
    var hex = c.Id.ToString("N");
    return $"did:sov:LY:{hex[^16..]}";
}
```

تُقرن دائماً الـ DID (هوية W3C خارج السلسلة) بعنوان على السلسلة (owner_address)، فتبقى عمليات التحقق بالهوية الذاتية (SSI) صالحة حتى بعد تدوير المحفظة.

٢.٨ نقل الملكية لاحقاً (TransferService)

بيع/إرث/هبة/قرار محكمة/تصحيح إداري ينقذها : POST /api/v1/property-nfts/{id}/transfer
نداء chain.TransferAsync ثم تحديث ثلاثة مواضع في حفظ واحد —
property_nfts (المالك الجديد + حالة transferred)، و ownership_history (صفّ جديد)،
و properties.owner_citizen_id (المالك القانوني). تُفرض نفس البوابة الأمنية (لا نقل لعقار
عليه نزاع نشيط) ويُطلب سبب صالح + ملاحظة إلزامية لقرار المحكمة/التصحيح.

٢.٩ التحقق العام (ownerOf + verify على السلسلة)

GET /api/v1/verify/{code} يقرأ بيانات العقار + تجزئة التحقق + الرمز، ثم يستعلم
chain.OwnerOfAsync(tokenId) ليُظهر المالك الحيّ على السلسلة الذي قد يختلف عن السجل
بعد

نقل خارجه — فيُكشف أي انحراف للعموم.

٢.١٠ المسارات (REST) وتسجيل الخدمات (DI)

الخدمة	المسار	الفعل
LicenseService.FinalApproveAsync (السكّ)	/api/v1/properties/{id}/final-approve	POST
سكّ جماعي	/api/v1/properties/bulk-final-approve	POST
TransferService.TransferAsync (النقل)	/api/v1/property-nfts/{id}/transfer	POST
NftsService.ListAsync (سجلّ الرخص)	/api/v1/property-nfts	GET
NftsService.ListHistoryAsync (سلسلة الحياة)	/api/v1/property-nfts/{id}/history	GET
NftsService.ListMyAsync (رخص المواطن)	/api/v1/me/nfts	GET

```
// Program.cs - Stub افتراضيًا IPFS حقن طبقة السلسلة و
builder.Services.Configure<BlockchainOptions>(builder.Configuration.GetSection(BlockchainOptionsSectionName));
builder.Services.Configure<IpfsOptions>(builder.Configuration.GetSection(IpfsOptionsSectionName));
builder.Services.AddSingleton<IBlockchainService, StubBlockchainService>();
builder.Services.AddSingleton<IIpfsService, StubIpfsService>();
```

٢.١١ حالة التنفيذ: Stub مقابل Ethereum — وكيف نتنقل للإنتاج

التصميم يعزل «ماذا» (سكّ/نقل/قراءة مالك) عن «كيف» (سلسلة فعلية أم بديل). للانتقال للإنتاج على إيثيريوم (موتّق حرفيًا في تعليقات BlockchainOptions.cs):

1. ضبط `Sarh:Blockchain:Mode = "ethereum"`.

2. تعبئة `RpcUrl` Infura/Alchemy أو عقدة Anvil/Hardhat محلية و `ContractAddress`

و `MinterPrivateKeyEnc` (مفتاح السكّ مُعَمّى بـ KMS — لا يُخزّن صريحًا أبدًا).

1. إضافة حزمة `Nethereum.Web3` وتنفيذ `EthereumBlockchainService`

`IBlockchainService`

(ينادي دوال العقد `ownerOf` / `safeTransferFrom` / `mint`).

1. تبديل سطر التسجيل في `Program.cs` إلى التنفيذ الجديد — لا يتغير أي شيء آخر في

LicenseService أو TransferService أو قاعدة البيانات أو الواجهة.

الشبكات المسموح بها في ck_nft_network جاهزة مسبقًا:

ethereum-mainnet , ethereum-sepolia , polygon-mainnet , polygon-amoy , hyperledger-fabric

٣. نموذج بيانات البلوكتشين (الترحيل 028)

```

-- property_nfts: الرابط (نظام التسجيل) بين العقار وهويته على السلسلة
CREATE TABLE property_nfts (
    id                UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    property_id       UNIQUEIDENTIFIER NOT NULL REFERENCES properties(id),
    token_id          NVARCHAR(80) NOT NULL,           -- uint256 (تفادي الفيض) كنص
    contract_address  NVARCHAR(80) NOT NULL,
    network           NVARCHAR(40) NOT NULL CHECK (network IN (...)),
    standard           NVARCHAR(24) NOT NULL DEFAULT N'ERC-721',
    owner_did          NVARCHAR(160) NOT NULL,          -- خارج السلسلة W3C هوية
    owner_address      NVARCHAR(80) NULL,              -- العنوان على السلسلة
    metadata_uri       NVARCHAR(255) NOT NULL,          -- ipfs://<cid>
    metadata_sha256    CHAR(64) NOT NULL,              -- المرساة المقاومة للتلاعب
    mint_tx_hash       NVARCHAR(80) NOT NULL,
    mint_block_number  BIGINT NULL,
    status             NVARCHAR(16) NOT NULL CHECK (status IN
        (N'pending', N'minted', N'transferred', N'burned', N'failed')),
    ...
);
-- رمز واحد لكل عقار (فهرس فريد مُرشَّح على الحالات النشطة)
CREATE UNIQUE INDEX ux_nft_property ON property_nfts(property_id)
    WHERE status IN (N'pending', N'minted', N'transferred');

-- ownership_history: (مثل audit_log) سلسلة حياة للإضافة فقط
CREATE TRIGGER tr_ownership_history_no_update ON ownership_history
INSTEAD OF UPDATE AS BEGIN RAISERROR(N'append-only; UPDATE not permitted.',16,1); END
CREATE TRIGGER tr_ownership_history_no_delete ON ownership_history
INSTEAD OF DELETE AS BEGIN RAISERROR(N'append-only; DELETE not permitted.',16,1); END

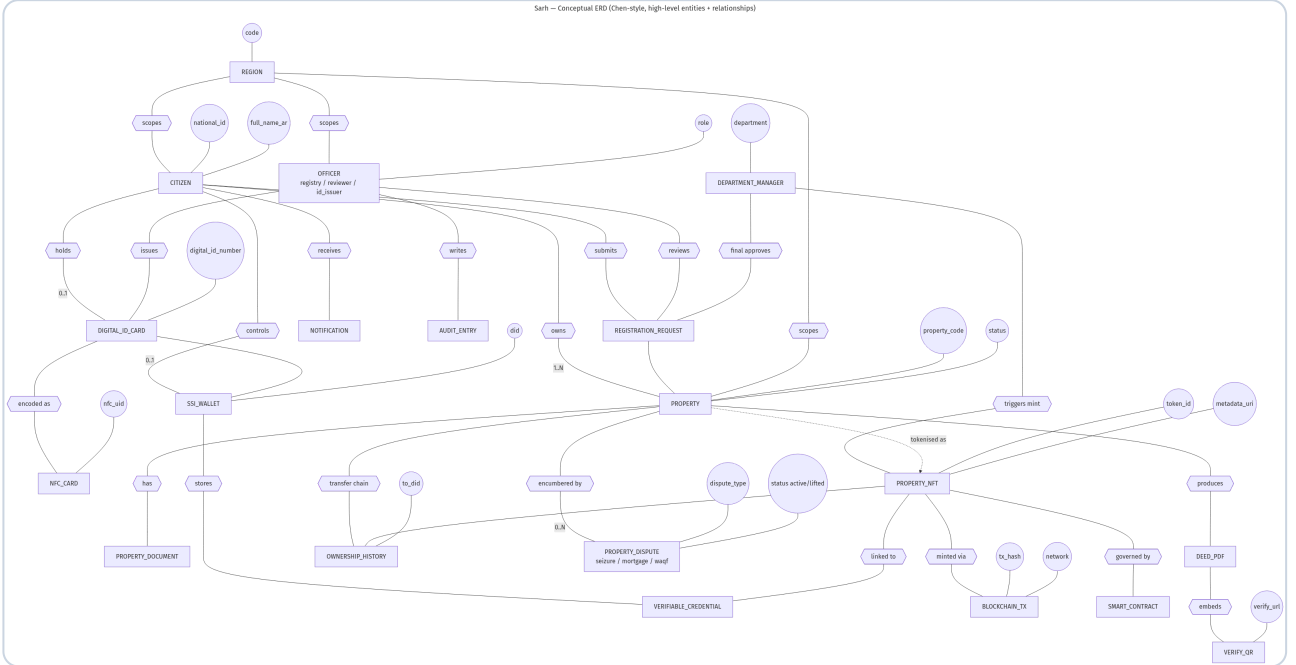
```

النقاط التي يُنصح بالمراجع بالتدقيق فيها: (أ) فصل الهوية القانونية عن مرآة السلسلة؛
 (ب) الإضافة-فقط على سلسلة الحياة بمشغلات (لا بالاتفاق فقط)؛ (ج) المرساة بالتجزئة
 (metadata_sha256) لمقاومة التلاعب؛ (د) فريدة الرمز لكل عقار عبر فهرس مُرشَّح.

٤. المخطط المفاهيمي (Conceptual ERD)

نظرة عالية المستوى (نمط Chen) للكيانات والعلاقات. كيانات البلوكتشين مُبرزة:

PROPERTY_NFT ، OWNERSHIP_HISTORY ، BLOCKCHAIN_TX ، SMART_CONTRACT

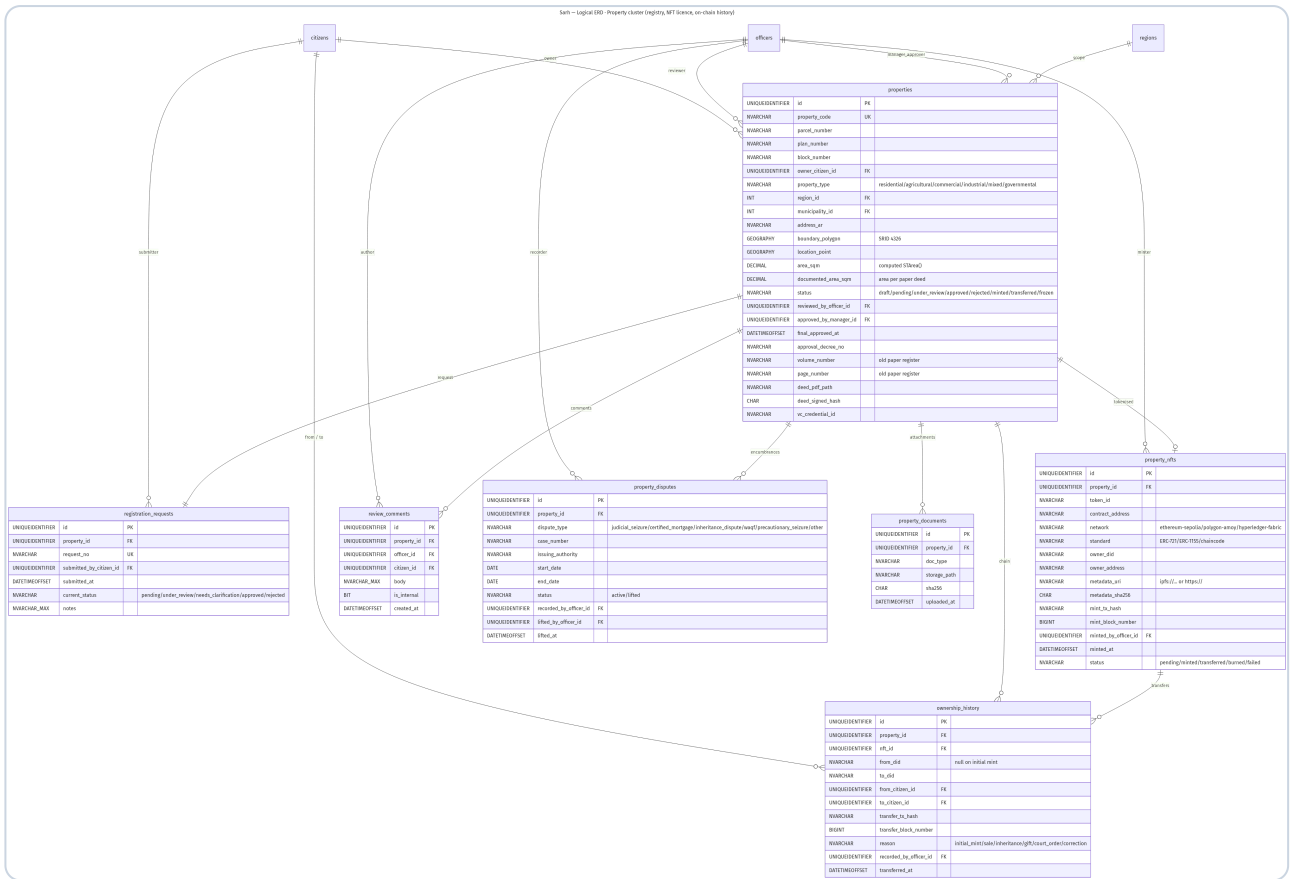


المخطط المفاهيمي — المصدر: docs/diagrams/conceptual-erd.mmd

علاقات البلوكتشين في هذا المخطط: DEPARTMENT_MANAGER -(triggers mint)→ PROPERTY_NFT ،
PROPERTY_NFT -(tokenised as)→ PROPERTY_NFT و PROPERTY_NFT -(minted via)→ BLOCKCHAIN_TX ،
PROPERTY_NFT -(governed by)→ SMART_CONTRACT و PROPERTY_NFT → OWNERSHIP_HISTORY ،
و PROPERTY_NFT -(linked to)→ VERIFIABLE_CREDENTIAL (سلسلة النقل)، و

٥. المخطط المنطقي (Logical ERD / UML)

المخطط الكامل بالجدول والمفاتيح (PK/FK/UQ). نظراً لحجمه، يُصيّر مقسماً إلى ثلاث لوحات (الهوية / العقارات / النظام) — مصدرها الموحد docs/diagrams/db-schema.mmd. اللوحة الخاصة بالعقارات أدناه تُظهر **جدولي البلوكتشين** property_nfts و ownership_history بكامل أعمدهما وعلاقتهما بـ properties و citizens و officers.



المخطط المنطقي — لوحة العقارات والبلوكشين (property_nfts + ownership_history). المصدر:

[docs/diagrams/db-schema.mmd](#)

اللوحتان الأخريان: الهوية الرقمية [docs/diagrams/db-schema-identity.png](#) ، والنظام/
التدقيق

[docs/diagrams/db-schema-system.png](#) . والمخطط المنطقي الكامل بصيغة نصية في [db-schema.mmd](#)

العلاقات الجوهرية للبلوكشين على المستوى المنطقي:

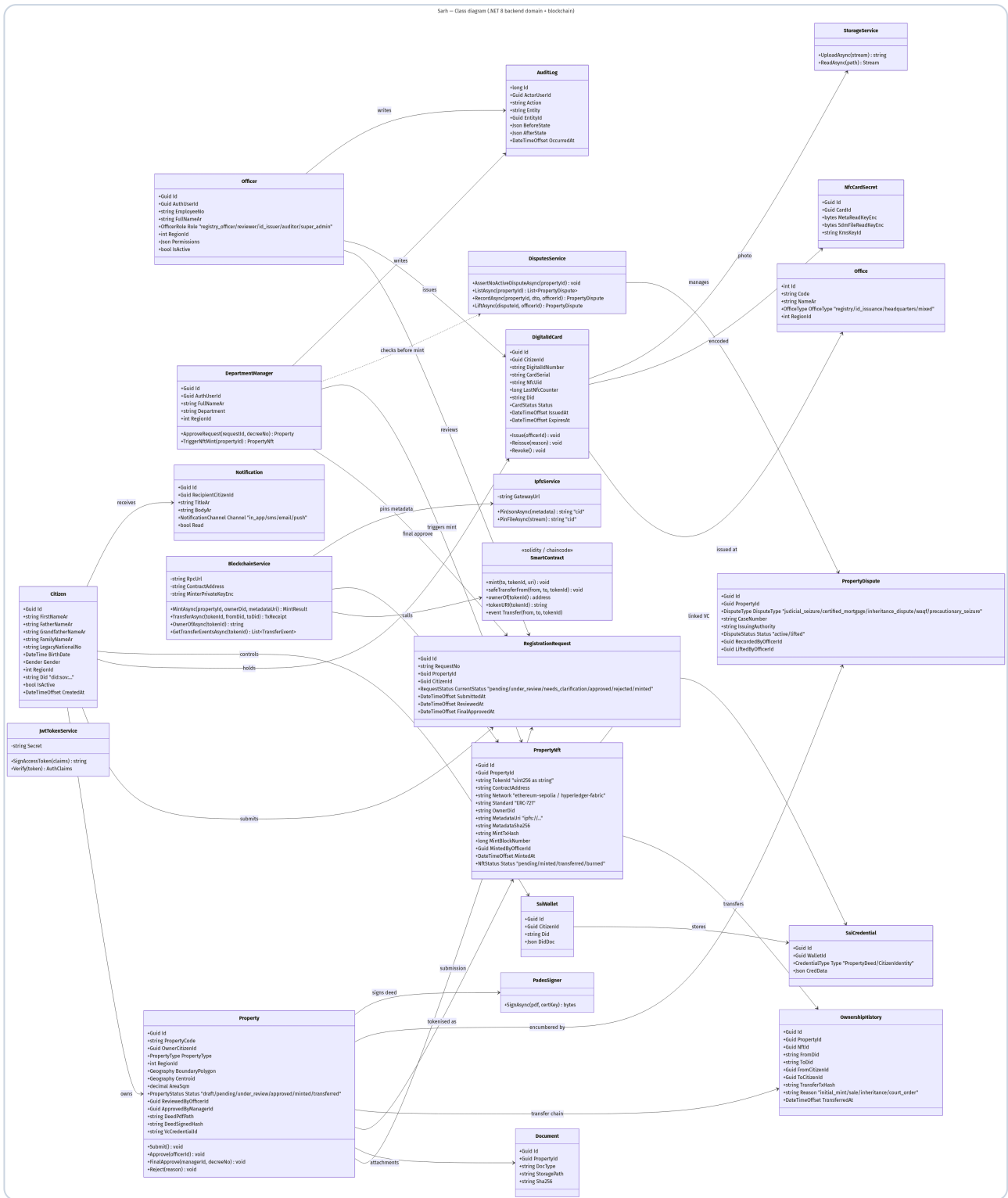
- `properties ||--o| property_nfts` — لكل عقار رمز واحد كحدّ أقصى («tokenised»).
- `property_nfts ||--o{ ownership_history` — لكل رمز سلسلة تحويلات.
- `officers ||--o{ property_nfts` — الموظف الساكّ («minter»).
- `citizens ||--o{ ownership_history` — طرفا التحويل (from / to).

٦. مخطط الفئات (Class Diagram)

نطاق الـ NET 8 + طبقة البلوكتشين. الفئات ذات الصلة:

, BlockchainService, SmartContract («solidity / chaincode»), IpfsService

.DepartmentManager.TriggerNftMint(...) إضافة إلى PropertyNft ، OwnershipHistory



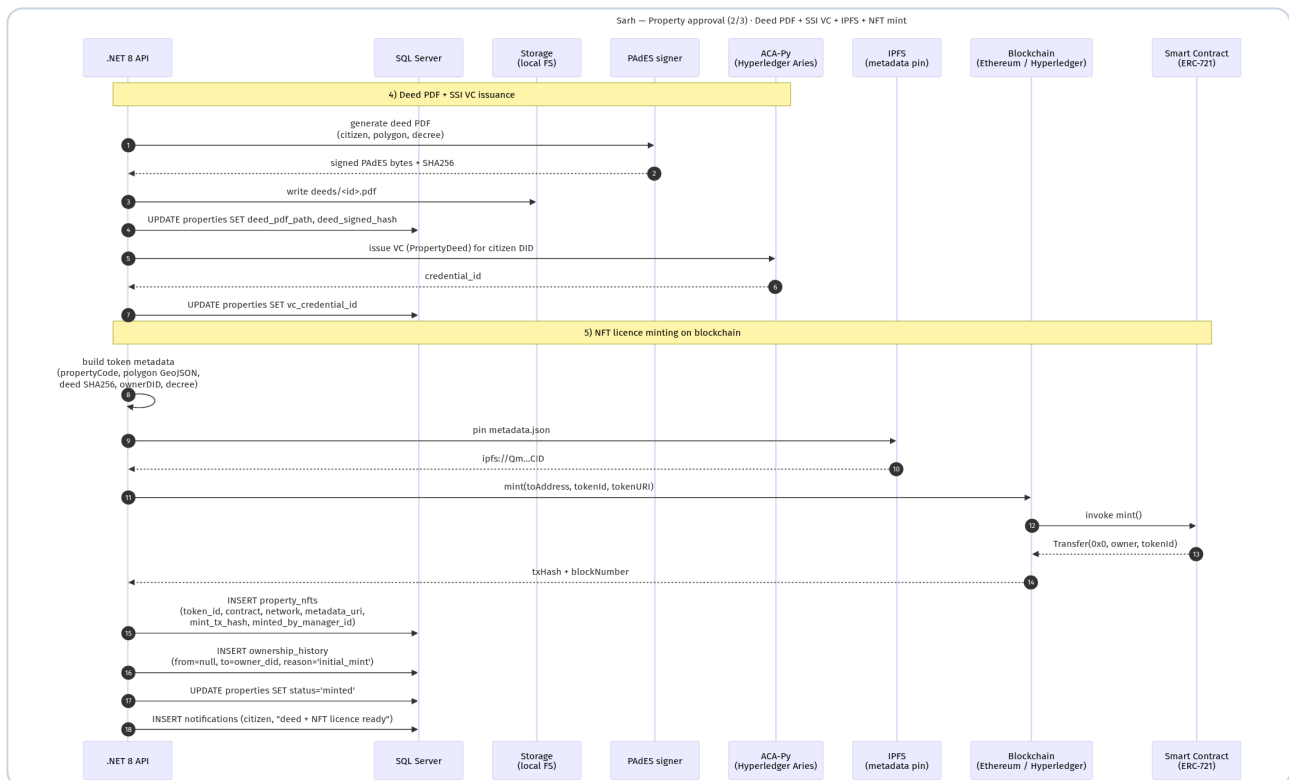
docs/diagrams/class-diagram.mmd: مخطط الفئات — المصدر:

تخطيط الفئات على الكود الفعلي:

فئة المخطط	الكود الفعلي
BlockchainService	واجهة IBlockchainService + تنفيذ StubBlockchainService (و EthereumBlockchainService مستقبلاً).
IpfsService	واجهة IIpfsService + تنفيذ StubIpfsService.
SmartContract	عقد ERC-721 الخارجي (Solidity) / Chaincode — تُنادى دواله .mint/safeTransferFrom/ownerOf.
PropertyNft / OwnershipHistory	كيانات EF Core على الجدولين.
DepartmentManager.TriggerNftMint	.LicenseService.FinalApproveAsync

٧. مخطط التسلسل لعملية السكّ (للسياق)

يوضّح المرحلتين ٤ و٥: إصدار الصكّ PAdES + اعتماد VC، ثم تثبيت الميادات على IPFS والسكّ على السلسلة وكتابة .property_nfts + ownership_history.



تسلسل السكّ — المصدر: docs/diagrams/sequence-property-mint.mmd

التسلسل الكامل من التقديم حتى التحقق متاح في `docs/diagrams/sequence-property-`
`approval.mmd`

٨. خلاصة للمراجع

- البلوكتشين ليس ناقصًا: الكود ونموذج البيانات والمسارات والمخططات كلها موجودة (الأقسام ٦-١).
- اعتماد الأرض على السلسلة = سكّ رخصة **ERC-721** عبر `LicenseService.FinalApproveAsync`،
مرساةً بميتاداتا IPFS مجزأة، ومسجلةً في `ownership_history` + `property_nfts`.
- التصميم قابل للإنتاج: طبقة سلسلة مجزأة (Stub اليوم، Ethereum/Nethereum بتبديل إعداد)،
مع فصل الملكية القانونية عن مرآة السلسلة، وسلسلة حيازة للإضافة-فقط، ومرساة مقاومة للتلاعب.

ملفات مرجعية: كود السلسلة `apps/api-dotnet/Blockchain/*` — خدمات السكّ/النقل
— `apps/api-dotnet/Workflow/{LicenseService,TransferService,NftsService}.cs`
المخطط
— `infra/mssql/migrations/028_property_nfts_ownership_history.sql` — الرسوم
`docs/diagrams/`

صّرح — توثيق النمذجة: المخطط المفاهيمي والمنطقي (UML) ومخطط الفئات

الوثيقة: توثيق نمذجة كامل لمشروع منصّة صّرح، يغطّي المخططات الثلاثة المطلوبة — المخطط المفاهيمي (Conceptual ERD)، والمخطط المنطقي (Logical ERD / UML)، ومخطط الفئات (Class Diagram) — مع قاموس بيانات كامل لكل الجداول، ومصفوفة تتبّع تربط النماذج بالكود وملفات الترحيل. مصدر الحقيقة للمخطط هو ترحيلات T-SQL المرقّمة في `infra/mssql/migrations/000...046.sql` وكيانات EF Core في `apps/api-dotnet/Data/Entities/`

• تمهيد ونطاق التوثيق

- **الاسم:** صّرح (Sarh) — منصّة السجل العقاري الليبي + إصدار الهوية الرقمية.
- **الجهة المالكة:** الرؤية الليبية للاتصالات والتقنية (LVCT).
- **المستخدمون:** المواطنون، موظفو السجل، المراجعون، مديرو الإدارة، موظفو إصدار الهوية، مدراء النظام، والمتحقّقون العموميون.
- **نطاق هذه الوثيقة:** نمذجة البيانات (المفاهيمي + المنطقي) ونمذجة البرمجيات (مخطط الفئات)،

مع قاموس بيانات تفصيلي. لتفاصيل المعمارية والأمان راجع `docs/project/03-architecture-and-design.ar.md` ولتفاصيل البلوكتشين راجع `docs/project/07-blockchain-erd-class-review.ar.md`.

منهجية النمذجة (المستويات الثلاثة)

المستوى	السؤال الذي يجب عنه	درجة التجريد	المخطط في صرح
المفاهيمي (Conceptual)	ما الكيانات الرئيسية وكيف ترتبط؟	عالي — بلا أنواع بيانات	المخطط المفاهيمي (نمط Chen)
المنطقي (/ Logical UML)	ما الجداول والأعمدة والمفاتيح؟	متوسط — مستقل عن محرّك بعينه	المخطط المنطقي (ER كامل) + قاموس البيانات
مخطط الفئات (Class)	كيف يُمثّل النطاق في الكود الكائني؟	تنفيذي — فئات .NET 8 وخدماتها	Class Diagram

١. نظرة عامة معمارية (سياق المشروع الكامل)

ثلاث طبقات رئيسية:

١. **الواجهة (Web):** تطبيق Angular 21 واحد (apps/web/) يجمع أدوار المواطن/الموظف / مُصدِر

الهوية/المدير/التحقّق خلف توجيه قائم على الدور، عربي أولاً (RTL). يوجد كذلك تطبيق Flutter للهاتف.

١. **الخلفية (API):** خدمة (apps/api-dotnet/) ASP.NET Core 8 موزّعة على وحدات:

Auth , Citizens , DigitalIdCards , Properties , Workflow (مراجعة/سكّ/نقل/صكّ),
Disputes , Map , Ssi , Blockchain , Notifications , Verify , Reports , Audit , Storage

١. **التخزين والخدمات:** SQL Server 2019/2022 (نوع geography للهندسة، فهرس عربي للبحث،

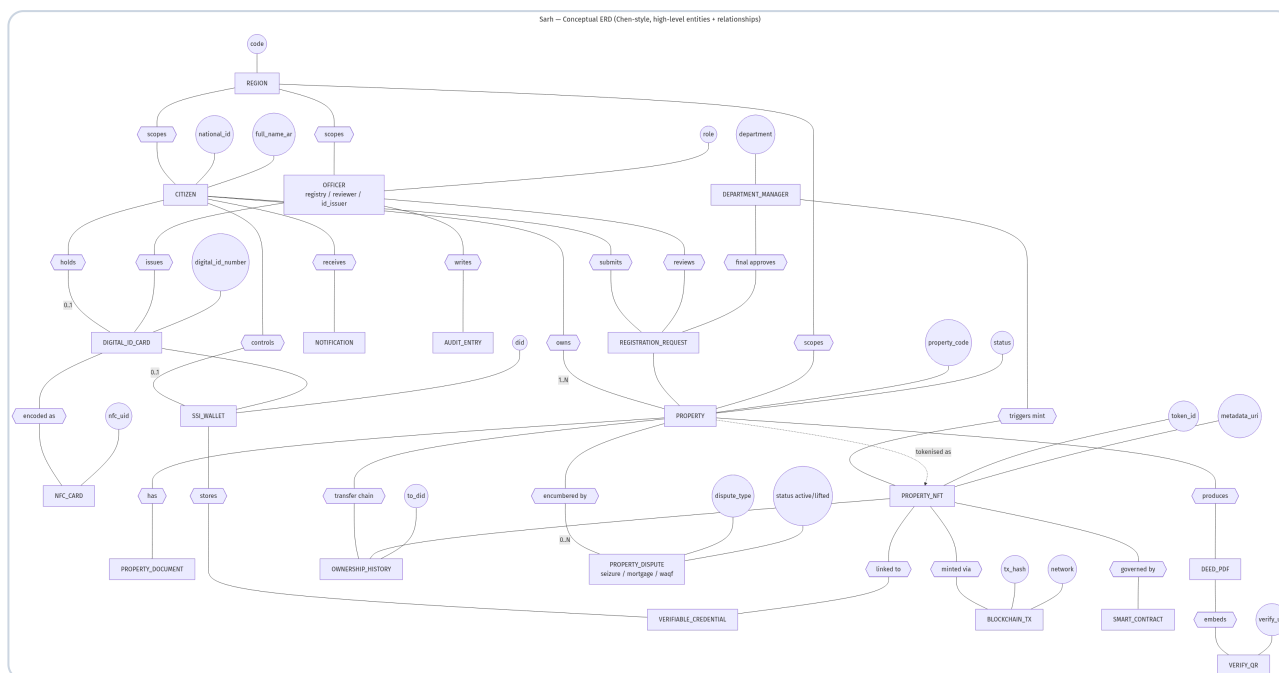
مشغّلات INSTEAD OF لسجّل التدقيق غير القابل للتعديل)، وكيل هوية ذاتية ACA-Py (Hyperledger

(Aries)، وطبقة بلوكتشين مجرّدة (Stub قابل للاستبدال بـ Ethereum)، وتخزين ملفات محلي.

تتوزّع كيانات قاعدة البيانات على **ثلاث عناقيد** نمذجها هذه الوثيقة: **عنقود الهوية** (الحسابات، المواطنون، الموظفون، البطاقات، المحافظ)، **وعنقود العقارات** (السجل، الوثائق، الرخص على السلسلة، سلسلة الحياة، النزاعات)، **وعنقود النظام** (الإشعارات، سجّل التدقيق).

٢. المخطط المفاهيمي (Conceptual ERD)

عرض عالي المستوى (نمط Chen): الكيانات مستطيلات، والعلاقات معيّّات، والسمات المفتاحية دوائر.



المخطط المفاهيمي — المصدر: docs/diagrams/conceptual-erd.mmd

٢.١ الكيانات الرئيسية

الكيان	الوصف
CITIZEN	المواطن الليبي (شخص طبيعي)؛ يحمل legacy_national_no لإعادة الإصدار بلا فقد بيانات.
OFFICER	موظف صرح؛ دوره يحدّد الصلاحيات (سجل/مراجع/مُصدر هوية/مدقق/مدير نظام).
DEPARTMENT_MANAGER	مدير الإدارة؛ الاعتماد النهائي وإطلاق سكّ الرخصة.
PROPERTY	العقار/القطعة المساحية؛ مساحته مشتقة من المضلع STArea().
REGISTRATION_REQUEST	مظروف دورة حياة طلب التسجيل وحالاته.
PROPERTY_DOCUMENT	مرفقات إثبات العقار (صورة موقع + كروكي إلزاميان).
PROPERTY_DISPUTE	عبء قانوني على العقار (حجز/رهن/نزاع ورثة/وقف)؛ صفّ نشيط يمنع السكّ والنقل.
DIGITAL_ID_CARD	بطاقة الهوية الرقمية NFC للمواطن.
NFC_CARD	الترميز الفيزيائي للبطاقة (NTAG 424 DNA + عدّاد متدحرج).
SSI_WALLET	محفظة الهوية الذاتية للمواطن (DID).
VERIFIABLE_CREDENTIAL	اعتماد قابل للتحقق (سكّ عقار/هوية مواطن).
DEED_PDF	سكّ PDF موقع PAdES يحوي QR للتحقق.
VERIFY_QR	رمز QR يشير إلى verify.sarh.ly/{code}.
PROPERTY_NFT	رخصة الملكية المسكوكة على السلسلة (ERC-721).
OWNERSHIP_HISTORY	سلسلة حياة للإضافة فقط (سكّ أولي + كل نقل).
BLOCKCHAIN_TX / SMART_CONTRACT	معاملة السلسلة والعقد الذكي الذي يحكم الرمز.
REGION	المحافظة (نطاق الصلاحية الجغرافي).
NOTIFICATION / AUDIT_ENTRY	الإشعارات وسجلّ التدقيق غير القابل للتعديل.

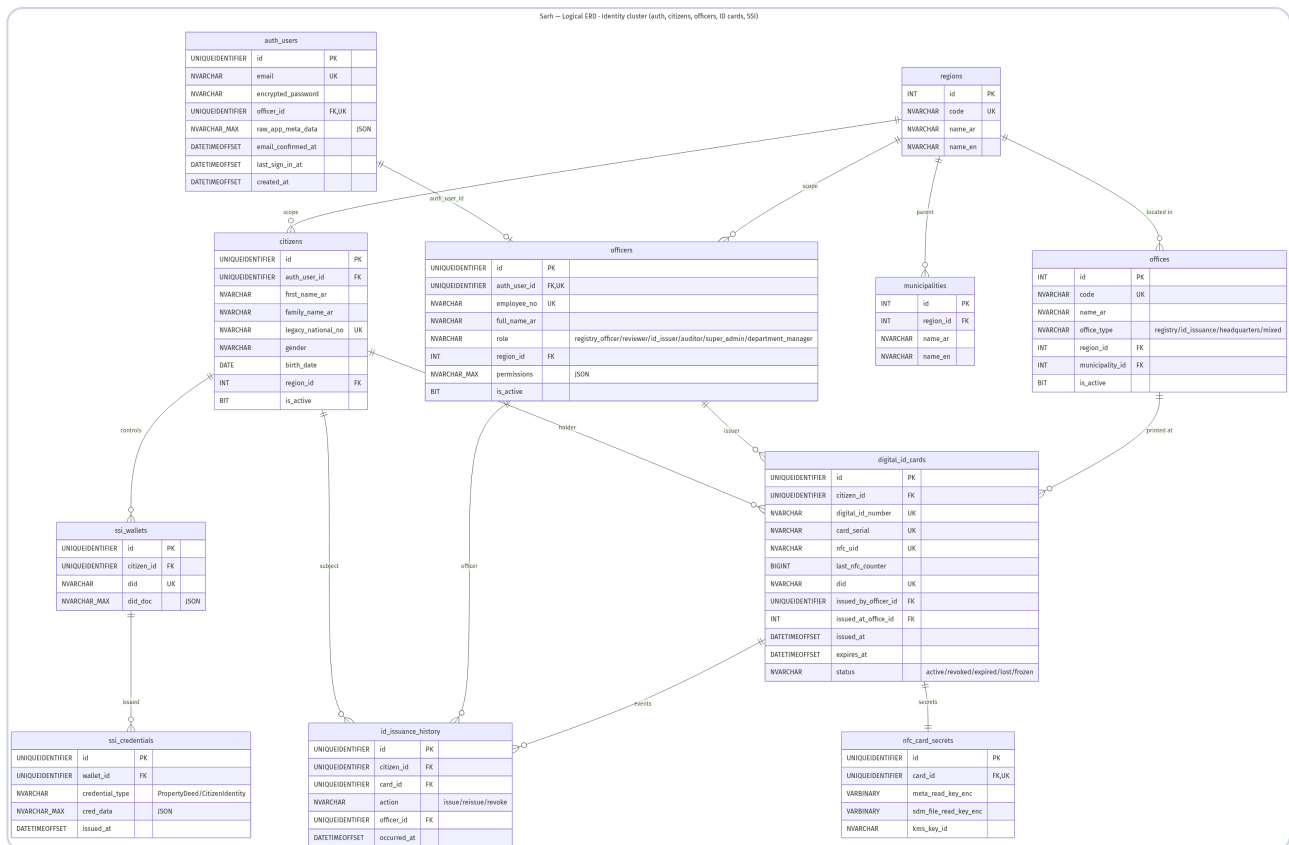
٢.٢ أهم العلاقات وتعدديتها (Cardinality)

العلاقة	الطرفان	التعددية
يملك	CITIZEN → PROPERTY	N..1
يحمل	CITIZEN → DIGITAL_ID_CARD	1..0
يتحكّم	CITIZEN → SSI_WALLET	1..0
يقدم	CITIZEN → REGISTRATION_REQUEST	N..1
يراجع	OFFICER → REGISTRATION_REQUEST	N..1
يُصدر	OFFICER → DIGITAL_ID_CARD	N..1
يعتمد نهائياً	DEPARTMENT_MANAGER → REGISTRATION_REQUEST	N..1
يُطلق السكّ	DEPARTMENT_MANAGER → PROPERTY_NFT	N..1
له وثائق	PROPERTY → PROPERTY_DOCUMENT	N..1
عليه عبء	PROPERTY → PROPERTY_DISPUTE	N..0
يُرمّز كـ	PROPERTY → PROPERTY_NFT	1..0
سلسلة نقل	PROPERTY/PROPERTY_NFT → OWNERSHIP_HISTORY	N..1
يُسكّ عبر	PROPERTY_NFT → BLOCKCHAIN_TX	N..1
مرتبط بـ	PROPERTY_NFT → VERIFIABLE_CREDENTIAL	1..0
يُرمّز فيزيائياً	DIGITAL_ID_CARD → NFC_CARD	1..1
يخزّن	SSI_WALLET → VERIFIABLE_CREDENTIAL	N..1
يحدّ نطاق	REGION → CITIZEN/OFFICER/PROPERTY	N..1

٣. المخطط المنطقي (Logical ERD / UML)

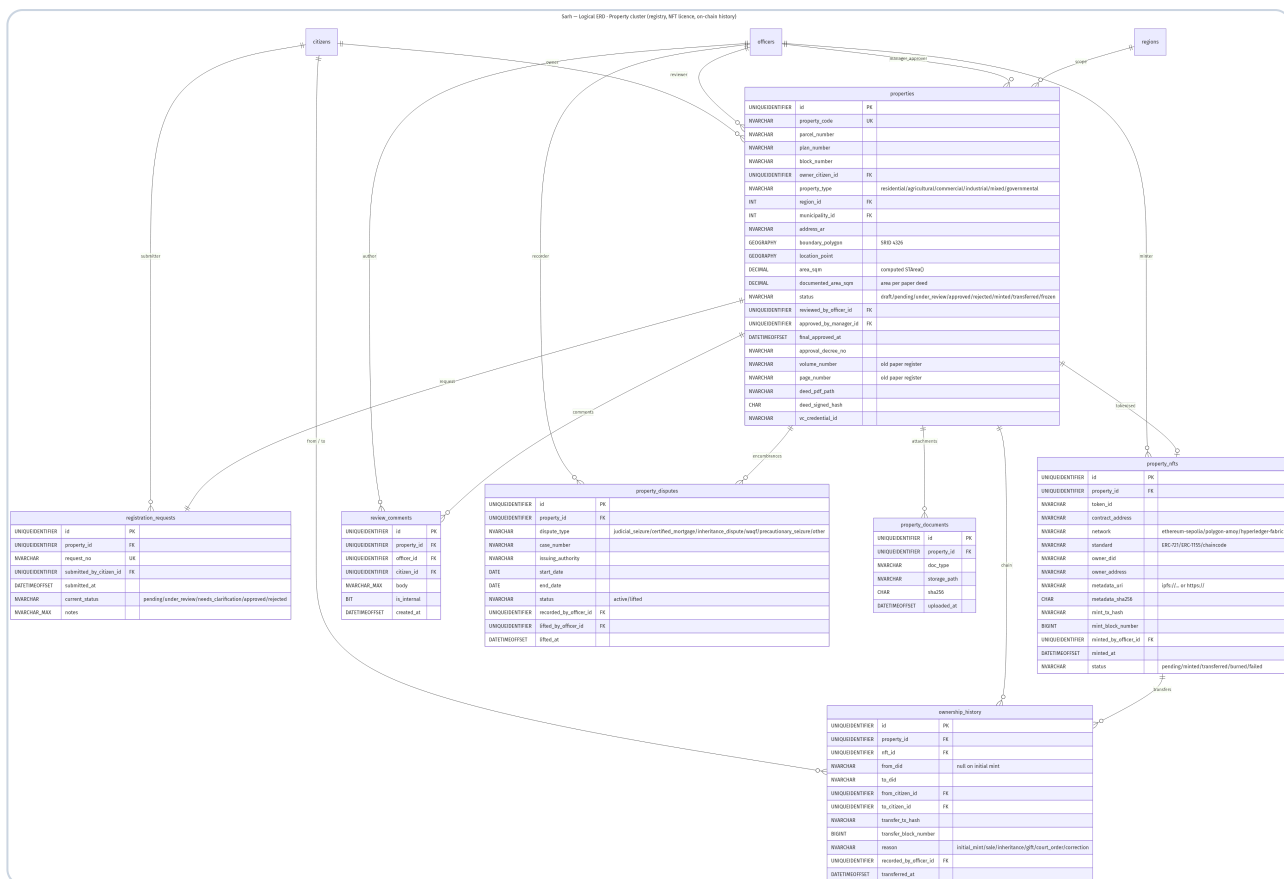
النموذج العلائقي الكامل بالجدول والأعمدة والمفاتيح (PK / FK / UK). نظراً لحجمه (٢٠ جدولاً) يُعرّض في ثلاث لوحات حسب العنقود؛ مصدرها الموحد `docs/diagrams/db-schema.mmd`.

٣.١ عنقود الهوية



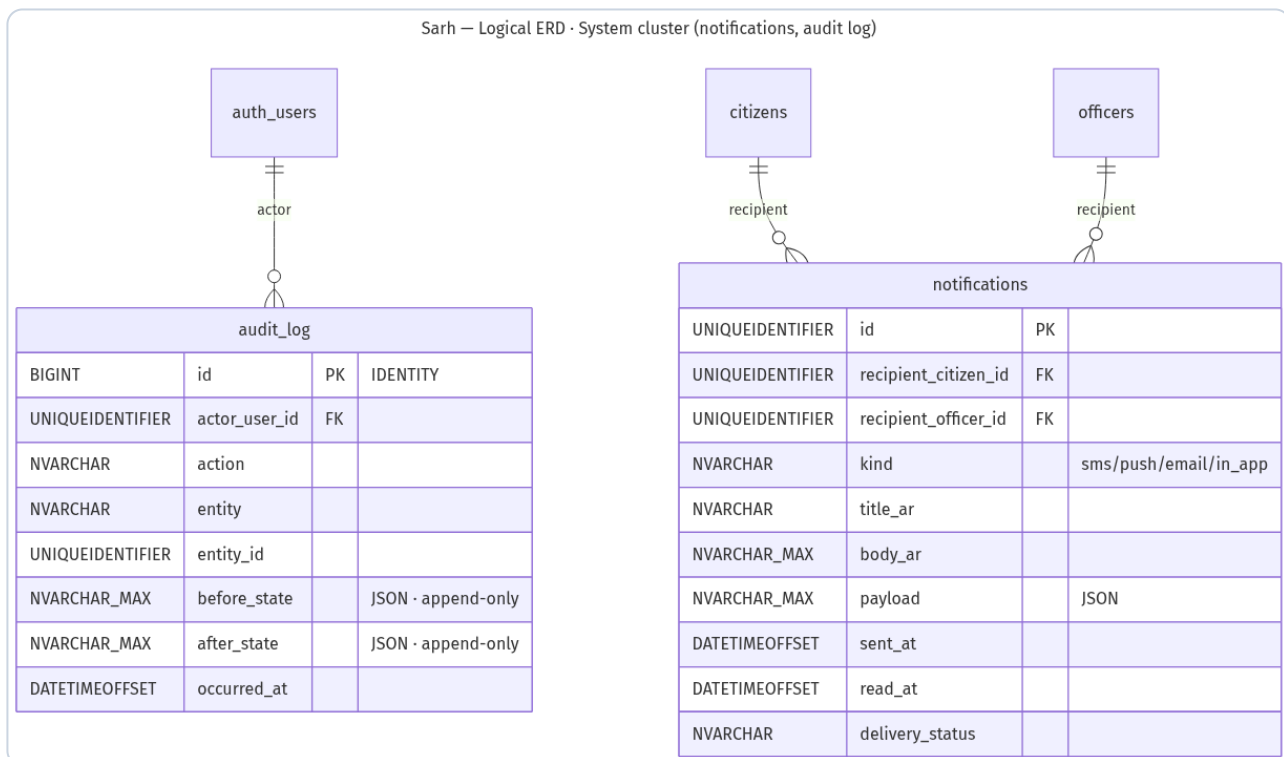
عقود الهوية: الحسابات، المواطنون، الموظفون، المناطق، المكاتب، البطاقات، المحافظ.

٣.٢ عقود العقارات والبلوكتشين



عقود العقارات: السجل، الطلبات، الوثائق، الرخص على السلسلة، سلسلة الحياة، النزاعات.

٣.٣ عقود النظام



عقود النظام: الإشعارات وسجل التدقيق (للإضافة فقط).

٣.٤ قاموس البيانات الكامل

اصطلاحات: كل المفاتيح الأساسية `UNIQUEIDENTIFIER DEFAULT NEWID()` (عدا `audit_log` و `regions / municipalities / offices` المُعرّفة `INT`). كل أعمدة الوقت `DATETIMEOFFSET(3)` بقيمة افتراضية `SYSDATETIMEOFFSET()`. كل كيان له `created_at / updated_at` (أسقطت من بعض الجداول اختصاراً). التعدادات `NVARCHAR(N) CHECK (col IN (...))` ، وحقول JSON `NVARCHAR(MAX)` بقيد `ISJSON()=1`. الحذف الناعم عبر `is_active = 0`. الفريدة على أعمدة قابلة للـ NULL تُنفذ بفهرس فريد مُرشّح (لا `NULL UNIQUE`).

auth_users — هوية المصادقة (سجلّ واحد لكل بيانات اعتماد)

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	المعرّف
email	NVARCHAR(254)	UK	بريد مُطبّع
encrypted_password	NVARCHAR(255)	NOT NULL	bcrypt (BCrypt.Net-Next)
officer_id	UNIQUEIDENTIFIER	FK→officers، فريد مُرشّح	رابط عكسي للحساب الموظّف (الترحيل 042)
raw_app_meta_data	NVARCHAR(MAX)	JSON	بيانات تطبيقية (الدور)
raw_user_meta_data	NVARCHAR(MAX)	JSON NULL	تفضيلات
email_confirmed_at / last_sign_in_at	DATETIMEOFFSET	NULL	تأكيد البريد/آخر دخول

citizens — المواطنون (أشخاص طبيعيون)

العمود	النوع	المفتاح/القيود	الوصف
id	UNIQUEIDENTIFIER	PK	
auth_user_id	UNIQUEIDENTIFIER	FK→auth_users، فريد مُرشَّح، NULL	NULL حتى التسجيل
first_name_ar / father_name_ar / family_name_ar	NVARCHAR(120)	NOT NULL	الاسم عربي
grandfather_name_ar	NVARCHAR(120)	NULL	
mother_name_ar	NVARCHAR(160)		اسم الأم
legacy_national_no	NVARCHAR(40)	فهرس فريد مُرَشَّح	الرقم الوطني الورقي — لا ينتهي
family_book_no	NVARCHAR	NULL	رقم كُتَيْب العائلة
gender	NVARCHAR(10)	CHECK (male, female)	
birth_date	DATE	NOT NULL	
birth_place / nationality / marital_status	NVARCHAR	NULL	
phone / email	NVARCHAR	فريد مُرَشَّح	
region_id	INT	FK→regions	محافظة التسجيل
municipality_id	INT	FK→municipalities، NULL	
address_ar	NVARCHAR	NULL	
is_active	BIT	DEFAULT 1	حذف ناعم

officers — موظفو صَرح

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
auth_user_id	UNIQUEIDENTIFIER	FK→auth_users, UK	حساب لكل موظف
employee_no	NVARCHAR(20)	UK	رقم وظيفي
full_name_ar / full_name_en	NVARCHAR		الاسم
role	NVARCHAR(40)	CHECK	super_admin, registry_officer, reviewer, auditor, id_issuer, department_manager
region_id / municipality_id	INT	FK	نطاق الصلاحية
permissions	NVARCHAR(MAX)	()JSON, ISJSON	خريطة صلاحيات دقيقة
is_active	BIT	DEFAULT 1	

regions / municipalities — مرجع جغرافي

الجدول	الأعمدة	الوصف
regions	id (INT PK), code (UK), name_ar, name_en	٢٢ محافظة ليبية، مبذورة مسبقاً
municipalities	id (INT PK), region_id (FK), name_ar, name_en	البلديات تحت كل محافظة

offices — مكاتب السجل/إصدار الهوية (الترحيل 040)

العمود	النوع	المفتاح/القيد	الوصف
id	INT IDENTITY	PK	
code	NVARCHAR(16)	UK	رمز المكتب
name_ar / name_en	NVARCHAR(128)		الاسم
office_type	NVARCHAR(24)	CHECK (registry, id_issuance, headquarters, mixed)	نوع المكتب
region_id / municipality_id	INT	FK, NULL	الموقع
address_ar / phone	NVARCHAR	NULL	
is_active	BIT	DEFAULT 1	

digital_id_cards — بطاقة الهوية الرقمية NFC

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
citizen_id	UNIQUEIDENTIFIER	FK→citizens	حامل البطاقة
digital_id_number	NVARCHAR(40)	UK	رقم الهوية الرقمية
card_serial	NVARCHAR(40)	UK	الرقم التسلسلي الفيزيائي
nfc_uid	NVARCHAR/CHAR	UK	NTAG لشريحة 424 DNA
last_nfc_counter	BIGINT		عداد SUN متدحرج — لا يتناقص
did	NVARCHAR(120)	UK	did:sov:LY:...
did_doc	NVARCHAR(MAX)	JSON	وثيقة DID
issued_by_officer_id	UNIQUEIDENTIFIER	FK→officers	المُصدر
issued_at_office_id	INT	FK→offices	مكتب الإصدار (الترحيل 042)
issued_at / expires_at	DATETIMEOFFSET		عادة +5 سنوات
status	NVARCHAR(20)	CHECK (active, frozen, lost, expired, revoked)	
photo_hash / data_hash	CHAR(64)	NULL	تجزئات مقاومة للتلاعب

البطاقة تحمل أيضاً رمز PIN مُجَرَّأً (الترحيل 031/033) للتحقق المحلي.

nfc_card_secrets — مفاتيح البطاقة (1:1 مع البطاقة)

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
card_id	UNIQUEIDENTIFIER	FK→digital_id_cards, UK	علاقة 1:1
meta_read_key_enc / sdm_file_read_key_enc	VARBINARY(MAX)		مفاتيح AES مُغَلَّفة
kms_key_id	NVARCHAR(80)		مرجع مادة KMS

id_issuance_history — سجلّ أحداث إصدار البطاقات

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
citizen_id / card_id	UNIQUEIDENTIFIER	FK	المواطن والبطاقة
action	NVARCHAR(20)	CHECK (issue, reissue, revoke)	الإجراء
reason	NVARCHAR	NULL	السبب
officer_id	UNIQUEIDENTIFIER	FK→officers	المنفّذ
occurred_at	DATETIMEOFFSET		

ssi_wallets / ssi_credentials — الهوية الذاتية (Aries)

الجدول	الأعمدة الرئيسية	الوصف
ssi_wallets	id (PK), citizen_id (FK), did (UK), did_doc (JSON)	محفظة المواطن ومرساة الـ DID
ssi_credentials	id (PK), wallet_id (FK), credential_type (CHECK: PropertyDeed, CitizenIdentity), cred_data (JSON), issued_at	الاعتمادات الصادرة JSON-LD

properties — السجل العقاري (القطعة المساحية)

العمود	النوع	المفتاح/القيود	الوصف
id	UNIQUEIDENTIFIER	PK	
property_code	NVARCHAR(40)	UK, NULL	يُصدر عند الاعتماد
parcel_number / plan_number / block_number	NVARCHAR		من المخطط المساحي
owner_citizen_id	UNIQUEIDENTIFIER	FK→citizens	المالك القانوني
property_type	NVARCHAR(20)	CHECK	residential, agricultural, commercial, governmental, industrial, mixed
region_id / municipality_id	INT	FK	الموقع الإداري
address_ar	NVARCHAR	NULL	عنوان حر
boundary_polygon	GEOGRAPHY	SRID 4326	حدود القطعة
location_point / centroid	GEOGRAPHY	محسوب	نقطة الموقع/المركز
area_sqm	DECIMAL(12,2)	محسوب STArea()	مساحة موثوقة من الخادم
documented_area_sqm	DECIMAL(14,2)	NULL, CHECK >0	مساحة الصكّ الورقي للمطابقة (الترحيل 039)
status	NVARCHAR(30)	CHECK	draft, pending, under_review, needs_clarification, approved, rejected, frozen, minted, transferred
submitted_at / reviewed_at / final_approved_at	DATETIMEOFFSET	NULL	طوابع الحالة
reviewed_by_officer_id	UNIQUEIDENTIFIER	FK→officers	المراجع
approved_by_manager_id	UNIQUEIDENTIFIER	FK→officers	المعتمد النهائي (الترحيل 028)
approval_decree_no	NVARCHAR(40)	NULL	رقم القرار

العمود	النوع	المفتاح/القيد	الوصف
volume_number / page_number	NVARCHAR(50)	NULL	إحالة السجل الورقي القديم (الترحيل 042)
deed_pdf_path	NVARCHAR(400)	NULL	مسار الصكّ
deed_signed_hash	CHAR(64)	NULL	SHA-256 للصكّ الموقع
vc_credential_id	NVARCHAR(80)	NULL	معرف اعتماد SSI

قاعدة الفريدة (3 § CLAUDE.md): لا يجوز لعقارين معتمدين تقاسم نفس المركز؛ وتقاطع المضلّعات يُطلق تحذيراً للمراجع لا منعاً صلباً (الصكوك الورقية قد تتقاطع شرعاً).

registration_requests — مظروف دورة حياة الطلب

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id	UNIQUEIDENTIFIER	FK→properties	العقار
request_no	NVARCHAR(40)	UK	REQ-YYYY-NNNN
submitted_by_citizen_id	UNIQUEIDENTIFIER	FK→citizens	مقدم الطلب
current_status	NVARCHAR(30)	CHECK	يعكس حالة العقار
notes	NVARCHAR(MAX)	NULL	

review_comments — محادثة المراجعة

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id	UNIQUEIDENTIFIER	FK→properties	
officer_id / citizen_id	UNIQUEIDENTIFIER	FK, NULL	الطرف الكاتب
body	NVARCHAR(MAX)		نص عربي
is_internal	BIT		ملاحظة داخلية للموظفين

property_documents — مرفقات الإثبات

العمود	النوع	المفتاح/القيود	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id	UNIQUEIDENTIFIER	FK→properties	
doc_type	NVARCHAR(40)		site_photo, koreky_certificate, ... , legacy_deed
storage_path	NVARCHAR(400)		تحت STORAGE_ROOT
sha256	CHAR(64)		تجزئة المحتوى
uploaded_at	DATETIMEOFFSET		

قاعدة الأدلة (CLAUDE.md §9): كل تقديم يجب أن يحمل على الأقل `site_photo` و `koreky_certificate` (كروكي)؛ وال API يرفض التقديم الناقص.

property_nfts — رخصة الملكية على السلسلة (الترحيل 028)

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id	UNIQUEIDENTIFIER	FK→properties، فريد مُرَشَّح	رمز واحد لكل عقار
token_id	NVARCHAR(80)	UK (network، contract، token)	uint256 كنص
contract_address	NVARCHAR(80)		عنوان العقد
network	NVARCHAR(40)	CHECK	ethereum-mainnet/sepolia، polygon-mainnet/amoy، hyperledger-fabric
standard	NVARCHAR(24)	CHECK، DEFAULT ERC-721	ERC-721، ERC-1155، chaincode
owner_did	NVARCHAR(160)		هوية المالك W3C
owner_address	NVARCHAR(80)	NULL	العنوان على السلسلة
metadata_uri	NVARCHAR(255)		ipfs://<cid>
metadata_sha256	CHAR(64)		مرساة مقاومة للتلاعب
mint_tx_hash	NVARCHAR(80)		معاملة السكّ
mint_block_number	BIGINT	NULL	
minted_by_officer_id	UNIQUEIDENTIFIER	FK→officers، NULL	السكّ (الترحيل 038)
minted_at	DATETIMEOFFSET		
status	NVARCHAR(16)	CHECK	pending، minted، transferred، burned، failed
last_reconciled_at	DATETIMEOFFSET	NULL	آخر مطابقة مع السلسلة

ownership_history — سلسلة الحياة (للإضافة فقط، الترحيل 028)

العمود	النوع	المفتاح/القيود	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id / nft_id	UNIQUEIDENTIFIER	FK	العقار والرمز
from_did	NVARCHAR(160)	NULL	NULL في السكّ الأولي
to_did	NVARCHAR(160)	NOT NULL	المالك الجديد
from_citizen_id / to_citizen_id	UNIQUEIDENTIFIER	FK→citizens	طرفا النقل
transfer_tx_hash	NVARCHAR(80)	NULL	معاملة النقل
reason	NVARCHAR(32)	CHECK	initial_mint, sale, inheritance, gift, court_order, correction
notes_ar	NVARCHAR(MAX)	NULL	
recorded_by_officer_id	UNIQUEIDENTIFIER	FK, NULL	الموظف المسجل
transferred_at	DATETIMEOFFSET		

مشغلات INSTEAD OF UPDATE/DELETE تمنع التعديل والحذف (مثل audit_log).

property_disputes — الأعباء القانونية (الترحيل 041)

العمود	النوع	المفتاح/القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
property_id	UNIQUEIDENTIFIER	FK→properties	العقار المُعبّأ
dispute_type	NVARCHAR(40)	CHECK	judicial_seizure, certified_mortgage, inheritance_dispute, waqf, precautionary_seizure, other
case_number	NVARCHAR(50)	NULL	رقم القضية
issuing_authority	NVARCHAR(150)	NOT NULL	الجهة المُصدِّرة
start_date / end_date	DATE	CHECK (end≥start)	مدّة العبء
status	NVARCHAR(24)	CHECK, DEFAULT active	active, lifted
recorded_by_officer_id	UNIQUEIDENTIFIER	FK→officers	المسجِّل
lifted_by_officer_id / lifted_at	UNIQUEIDENTIFIER / DATETIMEOFFSET	NULL	رفع العبء

صفّ نشيط يمنع كلاً من سكّ الرخصة والنقل في طبقة الخدمة حتى يُرفع العبء.

notifications — الإشعارات

العمود	النوع	المفتاح/ القيد	الوصف
id	UNIQUEIDENTIFIER	PK	
recipient_citizen_id / recipient_officer_id	UNIQUEIDENTIFIER	FK, NULL	المستلم
kind	NVARCHAR(20)	CHECK	in_app, sms, email, push
title_ar / body_ar	NVARCHAR		المحتوى
payload	NVARCHAR(MAX)	JSON	حمولة سياقية
sent_at / read_at	DATETIMEOFFSET	NULL	
delivery_status	NVARCHAR		حالة التسليم

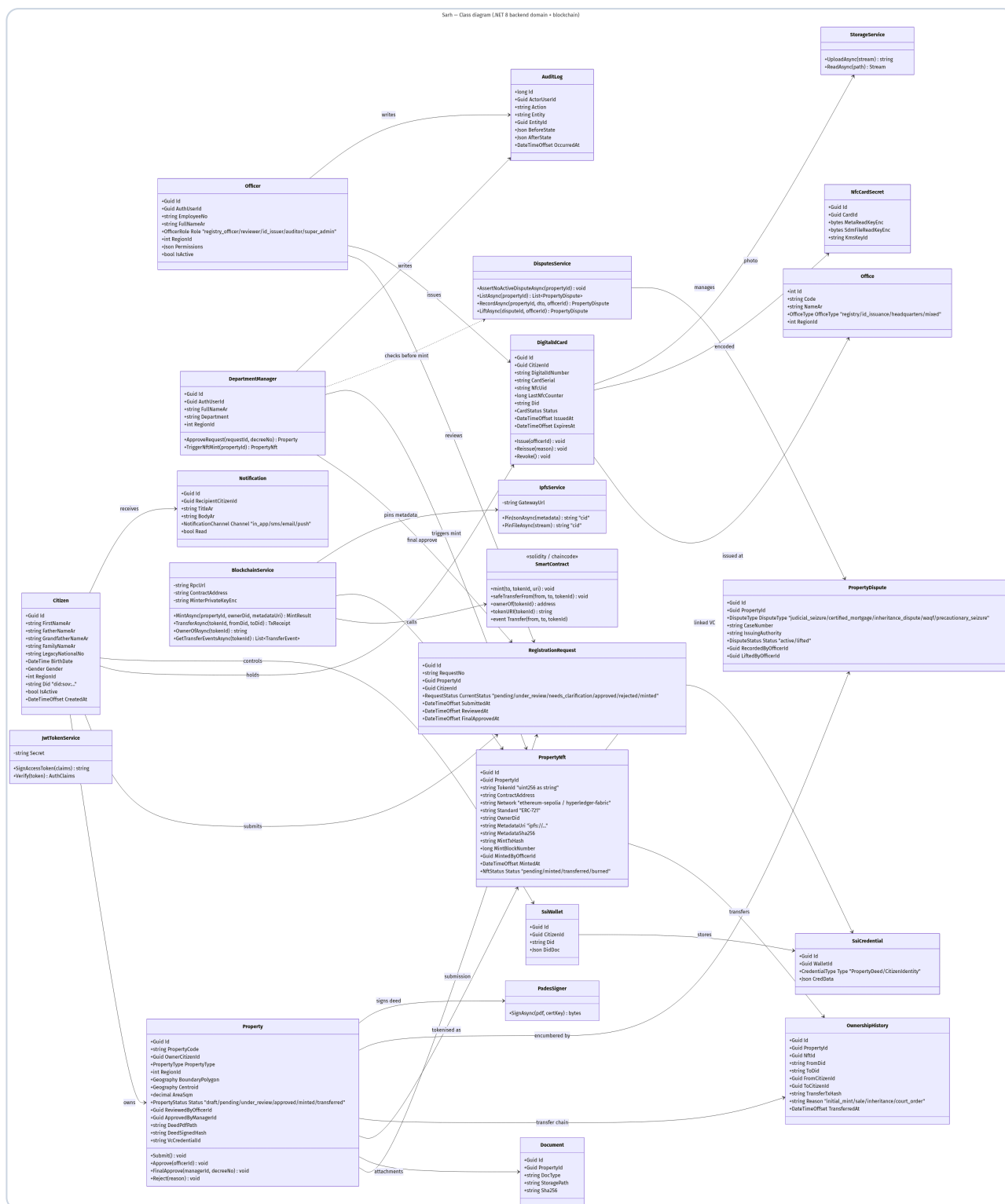
audit_log — سجلّ التدقيق (للإضافة فقط، الترحيل 011)

العمود	النوع	المفتاح/القيد	الوصف
id	BIGINT IDENTITY	PK	ترتيب زمني
actor_user_id	UNIQUEIDENTIFIER	FK, NULL	الفاعل (NULL للنظام)
action / entity	NVARCHAR		الإجراء واسم الجدول
entity_id	UNIQUEIDENTIFIER		الصفّ المتأثر
before_state / after_state	NVARCHAR(MAX)	JSON	الحالة قبل/بعد
occurred_at	DATETIMEOFFSET	DEFAULT (SYSDATETIMEOFFSET)	

مشغلات INSTEAD OF UPDATE/DELETE تمنع التعديل والحذف نهائياً.

٤. مخطط الفئات (Class Diagram)

نموذج النطاق الكائني في خلفية 8. NET، شاملاً كيانات البيانات وفئات الخدمات (السكّ، النقل، النزاعات، التخزين، التوقيع، المصادقة) والعناصر الخارجية (العقد الذكي، IPFS).



docs/diagrams/class-diagram.mmd: مخطط الفئات — المصدر

٤.١ الفئات الرئيسية ومسؤولياتها

المسؤولية	الفئة
كيانات الأشخاص والأدوار؛ المدير يطلق TriggerNftMint .	Citizen / Officer / DepartmentManager
العقار ومظروف دورة حياته.	Property / RegistrationRequest
مرفقات الإثبات.	Document
الرخصة على السلسلة وسلسلة حيازتها.	PropertyNft / OwnershipHistory
الأعباء القانونية وخدماتها (AssertNoActiveDisputeAsync) .	PropertyDispute / DisputesService
طبقة السلسلة المجردة، العقد الذكي، وتثبيت الميئاتااا.	BlockchainService / SmartContract / IpfsService
مكاتب السجل/الإصدار.	Office
بطاقة الهوية ومفاتيحها المٌغلّفة.	DigitalIdCard / NfcCardSecret
محفظة الهوية الذاتية واعتماداتها.	SsiWallet / SsiCredential
الإشعارات والتدقيق.	Notification / AuditLog
توقيع/تحقق JWT، التخزين، توقيع PDF بـ PAdES.	JwtTokenService / StorageService / PadesSigner

٤.٣ تخطيط الفئات على الكود الفعلي

الموقع في الكود	فئة المخطط
apps/api-dotnet/Data/Entities/*.cs (EF كيانات Core)	Citizen, Officer, Property, PropertyNft, OwnershipHistory, PropertyDispute, Office
apps/api-dotnet/Workflow/LicenseService.cs ((FinalApproveAsync	DepartmentManager.TriggerNftMint
apps/api-dotnet/Blockchain/IBlockchainService.cs + StubBlockchainService.cs	BlockchainService / SmartContract
apps/api-dotnet/Blockchain/IIpfsService + StubIpfsService.cs	IpfsService
apps/api-dotnet/Disputes/DisputesService.cs	DisputesService
apps/api-dotnet/Auth/JwtTokenService.cs	JwtTokenService
apps/api-dotnet/Storage/StorageService.cs	StorageService
apps/api-dotnet/Workflow/DeedSigningService.cs	PadesSigner

٥. مصفوفة التتبع (النموذج → الترحيل → الكود)

المجال	الترحيل (T-SQL)	الكيان/الخدمة (NET.)
الحسابات والمصادقة	042, 017	Auth/* , Data/Entities/AuthUser
المواطنون	036, 003	Citizens/* , Entities/Citizen
الموظفون والمكاتب	040, 005	Entities/Officer , Entities/Office
الهوية الرقمية والبطاقات	033, 031, 018, 004	DigitalIdCards/*
الهوية الذاتية SSI	022, 009	Ssi/*
العقارات	042, 039, 035, 019, 006, 046, 045, 043	Properties/*
الوثائق	007	Properties/ (uploads)
سير العمل والمراجعة	027, 021, 020, 008	Workflow/ReviewService.cs
الرخص على السلسلة + الحياة	038, 028	Workflow/LicenseService.cs , Blockchain/*
النزاعات	041	Disputes/DisputesService.cs
الإشعارات	010	Notifications/*
التدقيق	014, 011	Audit/*

٦. ملحق — إعادة توليد الرسوم وال PDF

- **مصادر الرسوم (Mermaid):** docs/diagrams/*.mmd — التصيير إلى PNG عبر [اسم] node docs/diagrams/render.mjs (يستخدم mermaid.ink ثم kroki.io احتياطياً).
- **المخطط المنطقي الكامل:** المصدر db-schema.mmd ، ويُصَيَّر مقسماً إلى ثلاث لوحات (db-schema-identity / db-schema-property / db-schema-system) لأن الرسم الكامل يتجاوز حدود خدمة التصيير.
- **هذه الوثيقة ك PDF:** node docs/print-modeling-docs.mjs → docs/project/Sarh-Data-Class-Models.pdf .

مصدر الحقيقة دائماً هو ترحيلات T-SQL في `infra/mssql/migrations/000...046.sql` وكيانات EF Core؛ المخططات وهذه الوثيقة مشتقة منها وتُحدّث عند تغيّر المخطط.